Product Requirements Document (PRD)

Service

Product *Multi-cloud Gateway*

Product Manager *Chris Muro*

Revision History

Date	Version	Description	Author
<mm/dd/yyyy>	1.0	*<description of change>*	*<name>*

Related Documents

Document Name / Description	Document Location

Table of Contents

What is a Product Requirements Document ?

The Product Requirements Document (PRD) describes a product or feature that addresses an opportunity or need. The PRD describes a particular product or feature release along the path of the product's strategy and roadmap, that clearly and unambiguously articulates the product's purpose, features, functionality, and behavior. How the requirements will be fulfilled are detailed in supplementary specifications by A&E, Operations, IT, etc.

Guiding Principles

The following are a few important principles to keep in mind as you write your PRD. These as well as the content in the "Product Requirements Process" section in the appendix are pieces that have been taken and edited from [How to Write an Effective Product Requirements Document (PRD)].

You are NOT the customer

Often, functionality that seems obvious and intuitive to those who design a product can seem obtuse and counterintuitive to potential customers and users who are exposed to it for the first time. A poor first impression can sour a potential customer on your product.

You must perform reality checks of your product with customers on an ongoing basis. Do they understand how it works? Will they use it? Will they value it? Customer feedback during development is invaluable.

Do NOT favor or push a specific solution

When writing requirements, you must specify what the product must do (the problem to be solved), NOT how the product will do it (the solution). It is important not to bias requirements toward one solution or another.

There are two pitfalls with solution-biased requirements.

First, engineers might make incorrect assumptions. There can be many different solutions to the same problem. Some will be better than others. Incorrect assumptions may exclude the best of those solutions, closing doors to innovation.

Second, you risk frustrating the development team. Your engineers are paid to come up with a solution, which they then must implement and maintain. They're the folks who will have to live with it and support it. Engineers want autonomy to be innovative. They tend to resent specifications that read like a recipe.

Ensure that every requirement in the PRD is clearly stating what the product must do but not dictating the solution itself.

Too little detail

While you want to give engineers and designers plenty of leeway to find the best solution, you still need to fully specify the product.

If there are gaps in your PRD, you may pay a heavy price down the road. Team members are likely to make assumptions to fill in the gaps. Different team members will likely make different assumptions. Problems no one anticipated may rear their ugly heads late in the game when it's far more expensive to correct them.

The best safeguard against this is to have the team review drafts of the PRD early and often. Address all questions and resolve them in the PRD. Plan on problems cropping up during development. Allocate time for addressing them. Update the PRD as they are resolved.

Too much detail

Too much detail can be just as bad as too little. If the PRD becomes a massive tome, team members will lose patience and won't read it. This can lead to more assumptions.

Remember that the PRD is a top-level document. Use the appropriate level of detail. If your product is highly complex you may need lower-level documents, like software requirements specifications, for each of your subsystems.

State requirements succinctly. Segregate explanatory text and keep that succinct as well. Add only what is necessary for context.

Feature overload

You want your product to appeal to as many customers and users as possible. So, it can be tempting to add features to meet everyone's needs and desires. Unfortunately, especially with high-tech products, adding features is often counterproductive.

Piling on features adds complexity to the product. That can make some features harder to find and lead to user confusion. An overabundance of features also drives up maintenance costs and customer support costs.

Be judicious in adding features. Exercise discipline. The tech world is littered with products that started off great, then became so bloated that customers abandoned them. They no longer served their purpose as well as they did when they were new and lean.

Engineering-driven requirements

Your engineering team may be enamored with an emerging technology they want to work with. They may be excited by a challenging problem they want to solve. This can lead them to overwhelm the product manager with requirements, effectively dictating the solution they want to build.

Product managers should listen to engineering needs but focus on value. They need to define a product that customers will buy and that's feasible to build, one that can be executed on time with a strong effort. They can't let engineers build whatever they want or something that can't be achieved within program's time constraints.

Marketing-driven requirements

Sales and marketing also want a say in product definition. They maintain close contact with their best customers. They have first-hand knowledge of what customers want, and they're charged with selling that.

One way this becomes a problem is with what are called "specials." Someone promises a customer a special version of your product with new features the customer requested. Typically, this is done in exchange for additional purchases or fees.

Specials create several issues.

First, the additional features create add complexity which can lead to customer confusion and frustration.

Second, you wind up with multiple versions of the product that you now must build, test, maintain and support. All that added task can bog down engineering and prevent Product from pursuing more profitable work.

Finally, customers often struggle to articulate what they need. The requested features may have been a stab at what they think they want. Once they receive their new features, users may find they don't meet their needs after all.

To avoid this pitfall, be disciplined. Evaluate the value of each proposed feature. Know when to walk away and be willing to do so.

Better yet, make sure your standard product is useful and compelling enough that customers can live without custom features. Get those valued customers involved in product concept and usability testing. Dig deep to get at what they really need.

1 Product Executive Summary

Multi-Cloud Gateway (MCGW), at its most basic form is a virtual routing service that allows customers to route between their Fabric Connections and Fabric Services. MCGW will play a critical role within Lumen's Multi-Cloud Networking strategy. The simplest use case for MCGW revolves around providing customers the ability to route between multiple Public Clouds at the Cloud OnRamps. Its value in this use case, is its ability to remove the complexity of routing between multiple Public Clouds, simplifying network design, and removal of additional latency caused by hair pinning traffic at a customer's premise.

Multi-Cloud Gateway is a consumer of Lumen Digital products. It provides customers the ability to put Layer 3 IP addressing on top of their Layer 2 connections. Thus providing routing intelligence that allows a customer to route to/from/between Layer 2 connections, Layer 3 Private IP (IP VPN VRF) and Public IP (Internet) connections.

2 Problem and Value Statements

The following are the customer and/or Lumen problem and value statements.

2.1 Customer Problem Statement

Customers are struggling with multi-cloud networking, interconnection and routing between all their cloud environments. Multi-Cloud Gateway (MCGW) will provide customers a simple to use virtual routing service that will allow them to easily interconnect their cloud environments utilizing private connectivity.

Customers using traditional applications and onsite infrastructure face several pain points impeding efficiency, costs, and performance. They require cloud-based applications and storage now for their business and AI needs and that becomes very complicated to manage efficiently.

A growing number of new businesses are born-in-cloud or have fully embraced the cloud and migrated all their resources into multiple clouds. These businesses have no traditional infrastructure, but they require Layer 3 services as they become multi-cloud and/or expand their own private and public network footprint. They require an ethereal solution that isn't tethered to a single cloud provider or reliant on equipment in specific physical premises. As more existing businesses become fully digitized, they are expected to evolve some of the same Layer 3 service requirements of their born-in-cloud counterparts.

Many of these customers have no prior experience in networking or are transforming into IT-lite organizations that are losing networking – particularly routing – expertise. Any solution should be easy-to-consume for a full range of user capabilities and needs, incorporating functionality and design knowledge that may have been assumed in prior Layer 3 service offerings.

All of these customers are digitizing businesses with different operations environments and cloud workflows. Integration of a solution will require support of a consistent API.

2.2 Customer Value Statement

MCGW will help customers who don't have presence within a data center to route between public clouds at the cloud on-ramps instead of backhauling each cloud connection to their premise. Customers will only need to have a single connection into the Lumen network to access all their public cloud environments, regardless of region and geography.

Lumens private connectivity fabric provides access to virtual routing services through a network built with Lumens dark fiber, waves, ethernet, and IP. This network spans multiple cloud platforms like AWS, Azure, Google, and Oracle allowing simultaneous use of public, private, and edge clouds from these CSPs. This approach mitigates risks and enhances performance by avoiding reliance on a single vendor. This helps simplify their multi-cloud network management leading to reduced CAPEX and OPEX costs and improved performance and scalability. {expand on and needs to call out ease of use and approachability. Ability to manage and understand and the ability to articulate that what we are building can fit both types of customers ala cloud and networking focused customers}

- Reduction in capital and operational expenses
  - Egress costs by utilizing private connectivity for their inter-cloud traffic instead of the Internet
  - Connectivity costs by not requiring backhaul of multiple cloud connections to their premise
  - Data center costs by not requiring presence in a colocation facility
- Increase of performance by routing at the cloud on-ramps
- Easy integration into modern operations environments

2.3 Segment Specific Statements

The problem and value statements for MCGW are common across all market sectors, with acceleration in importance as the Enterprise scales. The dominant percentage of all mid-market and large Enterprises now have workloads or projects in multiple public clouds, as well as the public sector. Lumen Problem and Value Proposition Statement – Why Lumen

MCGW is an important component of the corporate strategy goal for Lumen to capture 10% of the 3.8B multi-cloud market by 2028. As part of the initiative, Lumen Digital services including NaaS, Cloud Connect, and MCGW are to become a 300M ARR business by EOY 2027. MCGW and related Cloud Connections could capture up to 1/3 of that total business, or $100M annually.

Major demand drivers this product can opportunity capture through implementing a MCN infrastructure

- AI workloads
- Application and business resiliency
- Cybersecurity
- Further monetize Fabric Connections
- Generate additional revenue from the network

MCGW offers reduced latency through a fully meshed multi-cloud WAN solution and enables quick scaling.

MCGW eliminates the need for physical presence, allowing expansion to new regions and integration with other CSPs.

MCGW addresses latency and performance issues on legacy networks, reduces CAPEX and OPEX costs by removing the need for physical infrastructure, resources, and hardware, and simplifies cloud adoption.

MCGW removes the complexity of managing diverse architectures, technologies, protocols, and requirements, allowing for scalable growth without increased operational overhead.

While initial MCGW deployments are driven by IaaS uptake in enterprise digitization, MCGW is envisioned to become the cloud-delivered enterprise Layer 3 service of the future, providing connectivity for SaaS, PaaS, Financial Exchanges, and Extranets.

MCGW market success is tied to a set of engineering and infrastructure competencies: IP peering, cloud on-ramp density and distribution and a fabric interconnecting market leading Data Center sites, major metropolitan areas and cloud on-ramp sites. Lumen's acknowledged leadership in IP networking (transit and peering) and national fiber infrastructure give them the right-to-win in this product/service category.

2.4 Market and Competitive Outlook

The market for MCGW services is just developing in the US/NA and is dominated by large Data Center Operators (with inter-facility fabrics) like Equinix and smaller startups like Megaport and PacketFabric.[1]. These products provide the benchmarks for service delivery for their service category.

While exact attribution of their Cloud Router products (MCGW equivalents) to revenue are not separated in financial reporting of these entities, both Megaport and Equinix revenues for the combination of their MCGW functionality and associated cloud connections exceed $100M ARR with >30% CAGR in the US.

Other potential competitors include:

Digital Realty Trust (DRT), has their own inter-DC fabric and cloud on-ramps after a long partnership with Megaport. DRT's Virtual Router via their ServiceFabric offering would be a direct competitor to MCGW.

Coresite also has a significant cloud on-ramp business and multi-cloud routing capabilities through their partnership with Arrcus but weak marketing outside of their own resident customer base and financials obscured by their acquisition by American Tower.

Smaller international entities with developing footprints in the US like ConsoleConnect who do have a cloud router product but lower market presence and brand recognition.

2.5 Success Criteria

Existing products with the functionality of an MCGW are measured on two axis – total number of MCGWs in production yearly and the number of adjacent services that MCGW customers consume. The first axis combines both a revenue target (see above reference to overall goals), churn and retention measurements. The second combines adoption and satisfaction attributes.

For example, Megaport (the only competitor that publishes their product KPIs) grows the number of Megaport ~~Clo~~ud Routers at an annual rate of ~125/yr. MCR users generally consume ~17 'services'/MCR, which include cloud attachments, ports, the MCRs

[1] PacketFabric is expected to cease operations or be sold in the near future.

themselves and attachments to other services (eg IX, SaaS) or uplift 'services' associated with the MCR like Network Address Translation.

3 Initial Assumptions, Risks, and Dependencies

[Provide details for each assumption, risk, and dependency.

The values for the levels should be - Low, Medium, High

The values for the categories can be but not limited to - Product, Network, Systems, Marketing, Sales, Operations, Finance]

The following are the initial assumption, risks, and dependencies for the product.

3.1 Assumptions and Risk Level

[Underlying beliefs about user needs, market conditions, technical feasibility, or existing infrastructure that are being relied upon during product design and development.

Categories can be but not limited to - Product, Network, Systems, Marketing, Sales, Operations, Finance, Market Conditions, etc.

Indicate the level of risk for each assumption, if the assumption turns out to be incorrect.

Risk Levels should be - Low, Medium, High]

Assumption	Category	Risk Level	Details
No new hardware or software required for phase 1	Network	**High**	Current network devices, specifically AS3549 PE routers (VAR) can support the required functionality.
Current network infrastructure can support an increase in polling	Network	**High**	To be able to provide real-time view of route tables and telemetry data will result in an increase in polling of AS3549 PE routers (VAR).
NAT will not be available at initial release	Network	**Medium**	NAT support will most likely require new hardware to support NAT at scale.

Support for up to 1k routes per customer	Network	**Medium**	AS3549 PE routers (VAR) will be able to support up to 1k routes per customer. A single customer might have 1k routes in their combined route table, the routes might be spread across multiple PE routers.
Product funding will be distributed in 2 phases (seed funding vs 5-year bus case)	Finance	**Medium**	Initially we plan to use seed funding approved by Dave Ward on 5/15 to begin engineering development. Working w/FP&A to finalize 5 year business case view for MultiCloud Gateway which will include addt'l items outside of initial MVP (NAT, IPSec termination, FW, Cloud Exchanges, MCGW interface connected to public internet).
Fabric Port will be released prior to launch	Product	**High**	Terminology and design from a product perspective relies on the Fabric Port nomenclature. Majority of use cases rely on one or more Fabric Ports. MCGW is a consumer of other products, if the other products don't exist, MCGW won't be able to exist.
Fabric Connections will be released prior to launch	Product	**High**	Terminology and design from a product perspective relies on the Fabric Connection nomenclature. All use cases rely on attachment of a Fabric Connection to a MCGW Interface. MCGW is a consumer of other products, if the other products don't exist, MCGW won't be able to exist.
Public facing API redesign	Product	**High**	MCGW is being built with an API-first approach, its workflows and ability to provide its value relies on the API endpoints tied to other products. If those APIs don't exist nor perform in the same manner as MCGW's API endpoints the overall experience will be horrible from a user perspective.

3.2 Dependencies (A&E)

[Internal and external factors or elements that are needed for the product to function properly, meaning the development team needs to consider and potentially integrate these external components to fully realize the product's functionality.]

See 5.3 Release Dependencies

Impact Level	Category	Details
HIGH	Capacity Management	• Assuming customer traffic is using shared infrastructure NNIs to get to the cloud providers, we do not do a good job at managing those NNIs today. We have a solution, but it's large and was rejected 1.5 years ago so it hasn't been started.
HIGH	L2 Cloud Connectivity (Fabric Connections) PRD	Documentation within this PRD will be built to current/known specifications. Alignment to details that emerge in L2 Cloud Connectivity (Fabric Connections) PRD is critical
HIGH	API Development	Determination pending for API development impact. API specifications likely to be documented in a dedicated PRD which we need to maintain alignment to.
High	Fabric Port	With MCGW being a consumer of other Lumen Digital services, Fabric Port needs to be released prior to MCGW to ensure a cohesive and singular experience. • Fabric Connections needs to be released prior to MCGW • Public facing APIs need to be redesigned prior to MCGW
High	Fabric Connections	With MCGW being a consumer of other Lumen Digital services, Fabric Connections needs to be released prior to MCGW to ensure a cohesive and singular experience.
High	API Redesign	With MCGW being a consumer of other Lumen Digital services, public facing API needs to be redesigned to follow the OpenAPI 3.0 Specification and released prior to support an API-First approach.

4 Product Definition and Requirements

4.1 Product Principles

- Development with an API-first approach
- Build smarter, and more flexible user interfaces that allow consumers to build services in an automated fashion - particularly in the cloud. There is absolutely nothing that can be done in the UI that can't also be done in the API. Our API is the tool for large scale multi-cloud networking creation and scaling of customer workloads.
- The UI will follow as a flexible and modular framework that presents the API in a human friendly way
- Offer opt-in high-availability deployment options in a cloud-friendly “zone” concept: regional zones for services and local zones for physical connections (ports). These should be easy to consume (e.g. if the customer choses a multi-zone deployment of MCGW in a region, tooling should either automatically build connections to the redundant service points or aid the customer in doing so)
- Don’t dictate but rather empower the customer to determine how best to manage their connectivity given the tools, simplicity, and reliability of our network
- Use automation and provisioning capabilities to manage the network and protect it from user experience impacts
- Alignment on terminology and vocabulary to ensure clear and concise communication and understanding among customers in a ‘just enough’ detailed fashion to help improve understanding and mitigate confusion

4.2 Product Overview and Main Use Cases

[Provide an overview (product vision) and describe each of the main use cases.]

MCGW provides customers a virtual routing service, that can be used to route between Fabric Connections, Fabric Services, and Fabric Ports. The idea behind MCGW is to provide customers the experience of a physical device. This will be achieved by making the workflows, pieces, and terminology of MCGW to be like the steps of adding new connections to a router or gateway.

When adding a new connection to a physical device, one would take a Layer 2 connection from a Service Provider and plug it into an interface/port. Once the Layer 2 connection is attached to the device, one would then configure the interface with an IP address. After the IP address is configured, one would then configure the routing protocol being used on the device to include the new interface to allow traffic to route between the new connection and other existing connections.

With MCGW, the experience will follow the same steps. One will provision a Layer 2 connection, which will be a Fabric Connection. When provisioning the connection, one will indicate that one end of the connection should be attached to a MCGW Interface, like plugging a patch cable into a physical device. Once the Fabric Connection is attached to the interface, one will then configure an IP address on the MCGW Interface. After an IP address is configured on the MCGW Interface, one will be able to configure the routing protocol to allow traffic to route between the Fabric Connection and other existing Fabric Connections.

The experience described above does not go into intricate detail nor list out all the potential possibilities with MCGW.

Each MCGW is defined as an instance which maps to a global VRF within the AS3549 Lumen network. Fabric Connections are attached to a local VRF that is configured on the source or destination Metro VAR for the Fabric Connection. These local VRFs are then stitched to the MCGW instance (global AS3549 VRF) through route leaking by using import/export statements to insert routes into the MCGW instance.

All MCGW instances should utilize ASN1 as their default ASN.

Phase 1 Scope

Expected functionality will focus on layer 3 cloud to cloud connections where a customer with multiple cloud regions needs to connect them together using fabric ports and virtual connections and the MCG provides the virtualized intermediary peering entity, sessions and API interface to manage those interconnections.

In Phase 1, customers will have different levels of control over routing and policies including the ability to establish and control BGP peering (including BGP multi-hop support), IP prefix filtering and limiting for BGP sessions, BFD session fault detection and a limited set of route attribute controls that can influence route preference (eg MED) as well as default route configuration.

- Availability zones will not be supported in Phase 1
- Level of technical detail supported for Phase 1 platform (are we targeting everything from kindergarten level to PhD level or targeting a segment of that user base?)
    - i.e. preset basic/standard configs plus ability to expand features and manipulate data/options to low level of technical knowledge

4.3 Terminology

Multi-Cloud Gateway

Virtual Routing Instance assigned to a customer, which maps to a global (network-wide) VRF within AS 3549.

Multi-Cloud Gateway Interface

Virtual Interface that a Fabric Connection is attached to and where the Layer 3 addressing and BGP peering is configured. It is nothing more than a pointer in a backend database, it does not represent a physical interface.

4.4 Product Functionality and Features (PM and A&E)

[Provide details on how the product should function and what features the product should have. This is where the details of what the product can and/or cannot do.

Product Managers are responsible for the first 3 columns, and A&E are responsible for the column on the right.

Functionality refers to the specific actions and capabilities a product can perform, while "features" are the individual components or elements that enable those functionalities, essentially describing what the product can do from a user perspective, whereas functionality explains how it does it.

Key points to remember:

***Functionality**: Describes the overall behavior and purpose of a feature, including its inputs, outputs, and expected interactions with the user.*

***Features**: Represents a distinct, visible part of the product that provides a specific capability or benefit to the user.*

Example:

***Functionality**: "The user should be able to search for products on the website by entering keywords."*

***Feature**: "A search bar on the homepage with autocomplete suggestions for product names."*

These details are critical for engineering and others to understand what is being developed and what it should look like.]

Priority	Functionality	Feature ID	Feature
High	Users should be able to include a session token within the header of an API request.	LDVS-445	Utilizing a session token to authorize a request should be supported when included within the header of a request. Bearer Authentication should be supported.
High	Users should be able to login and request a session token using their username and password.	LDVS-446	An API endpoint should exist and be supported for login and requesting a session token. The endpoint should support username and password to authenticate the request. Path - /auth/login

			• Operation - POST
			• Request Body Schema
			  o Object - username
			    ▪ Type - string
			    ▪ Required - true
			  o Object - password
			    ▪ Type - string
			    ▪ Required - true
			
			Response should include:
			• Object - session_token
			  o Type - string
			• Object - token_expiration
			  o Type - string (date-time)

High	Users should be able to logout by sending a request to an API endpoint.	LDVS-447	The ability to logout through an API request that invalidates the session token within the hesder of the request should be supported. Path - /auth/logout Operation - GET
High	Users should be able to include an API key within the header of an API request.	LDVS-448	Utilizing an API key to authorize an API request should be supported when included within the header of a request. API keys should be supported.
High	Users should be able to create an API key.	LDVS-449	An API endpoint should exist and be supported for creating an API key. Path - /admin/api-keys/internal • Operation - POST • Request Body Schema o Object - key_name

			▪ Type - string ▪ Required - true o Object - key_expiration ▪ Type - integer (number of seconds) ▪ Required - false Response should include: • Object - token ○ Type - string • Object - key_name ○ Type - string • Object - api_key_id ○ Type - string • Object - token_expiration ○ Type - string (date-time)
High	Users should be able to view individual API keys.	LDVS-455	Viewing of an individual API key by referencing the api_key_id should be supported. Path - /admin/api-keys/internal/api_key_id} • Operation - GET • Parameter - api_key_id o Type - string o Required - true Response should include: • Object - token ○ Type - string • Object - key_name ○ Type - string • Object - key_uid ○ Type - string

			• Object - token_expiration ○ Type - string (date-time)

High	Users should be able to view all API keys.	LDVS-450	Viewing API keys should be supported.
			Path - /admin/api-keys/internal
			• Operation - GET
			Response should include:
			• Object - token
			  ○ Type - string
			• Object - key_name
			  ○ Type - string
			• Object - key_id
			  ○ Type - string
			• Object - token_expiration
			  ○ Type - string (date-time)

High	Users should be able to delete an API key.	LDVS-451	Deleting an API key should be supported by referencing the api_key_id.
			Path - /admin/api-keys/internal/{api_key_id}
			• Operation - DEL
			• Parameter - api_key_id
			  ○ Type - string
			  ○ Required - true
			Response should include:
			"{api_key_id}, has been deleted"

High	Users should be able to receive the state of a resource through each resources respective API endpoint. There should	LDVS-452	All resource API endpoints should support and include an object named "state" to indicate the moment in time (when the request was received) state of the resource. Multiple

	be multiple values used to indicate state.		values should be supported; values should be determined based on the type of resource and type of request.
			• Request Body Schema
			  ○ Object - state
			    ▪ Type - string
			    ▪ Read-Only - true
			The value for the "state" object in the endpoint should be set to one of the following determined by the backend workflow and request type.
			• Active (GET, POST, PATCH)
			• Disabled (GET, POST, PATCH)
			• Deleting (DEL, GET)
			• Provisioning (POST)
			• Updating (PATCH)
			• Resetting (PUT)

High	Users should receive the following information in a successful response to a POST, GET, PATCH, PUT, and DEL request.
	All endpoints:
	• Customer UID
	• User UID
	• Provision Date/Time
	• Last Modified Date/Time
	• Provisioning User
	• Last Modified User
	Price impacting endpoints:
		LDVS-454	A default set of read-only objects should be supported in each API endpoint. These objects should be included in a successful response to a POST, GET, PATCH, PUT, and DEL request.
			• Operation - POST, GET, PATCH, PUT, DEL
			• Request Body Schema
			  ○ Object - customer_uid
			    ▪ Type - string
			    ▪ Read-Only - true
			  ○ Object - user_uid
			    ▪ Type - string
			    ▪ Read-Only - true
			  ○ Object - time_provisioned
			    ▪ Type - string

	• Billing Account UID • Billing Status • Billing Start Date/Time • Billing Term • MRC		▪ Read-Only - true o Object - time_modified ▪ Type - string ▪ Read-Only - true o Object - user_provisioned ▪ Type - string ▪ Read-Only - true o Object - user_modified ▪ Type - string ▪ Read-Only - true o Object - billing (when a billable resource) ▪ Type - object • Object - account_uid o Type - string o Read-Only - true • Object - active o Type - boolean o Read-Only - true • Object - time_started o Type - string o Read-Only - true • Object - term o Type - string o Enum - "Hourly", "1M", "12M", "24M", "36M", "48M" o Read-Only – true
High	Users should receive all objects defined within the API endpoint along with their respective values in JSON format, when	LDVS-458	When a 201-response code is sent, returning all objects associated to the endpoint and their respective values within the response in JSON format should be supported.

LUMEN

	a POST request to an endpoint returns a 201-response code.		
High	Users should receive all objects defined within the API endpoint along with their respective values in JSON format, when a GET request to an endpoint returns a 200-response code.	LDVS-472	When a 200-response code is sent, returning all objects associated to the endpoint and their respective values within the response in JSON format should be supported.

High	Users should be able to provision a new MCGW instance.	LDVS-460	Provisioning of a new MCGW instance should be supported.
			Path - /mcgw/v1/gateways
			• Operation - POST
			• Request Body Schema
			  o Object - name
			    ▪ Type - string
			    ▪ Required - false
			  o Object - asn (see MCGW-ID-06)
			    ▪ Type - integer
			    ▪ Required - false
			  o Object - mcgw_tier
			    ▪ Type - string
			    ▪ Required - true
			  o Object - description
			    ▪ Type - string
			    ▪ Required - false
			There should not be a requirement to set the name during provisioning, the object should be optional. If a name is not provided during provisioning the name should default to the MCGW instance identifier.

High	There should be a mechanism for tracking the total aggregate bandwidth consumed by a MCGW instance.	LDVS-461	Ability to track and record the total aggregate bandwidth of an instance should be supported. This value will be used to determine when an instance requires a tier level upgrade. Total aggregate bandwidth is calculated by totaling the bandwidth of all Fabric Connections attached to a MCGW Interface. The value does not indicate the amount of throughput of an instance. For example: MCGW instance has four 1Gbps Fabric Connections attached, this would result in a total aggregate bandwidth of 4Gbps.

High	Users should be able to get/view the current total aggregate bandwidth of a MCGW instance.	LDVS-462	Ability to get/view the current total aggregate bandwidth of a MCGW instance should be supported. The value should be indicated in bits
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - GET
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Response Body Schema
			  o Object - total_aggregate_bw
			    ▪ Type - integer

High	Users should be able to select a tier level for the MCGW instance. See pricing section for tier levels.	LDVS-463	When provisioning a MCGW instance the ability to indicate what tier (price impacting) level to assign to the instance should be supported. Path - /mcgw/v1/gateways • Operation - POST • Request Body Schema o Object - mcgw_tier

			▪ Type - string ▪ Required - true

High	Users should be able to provide a description for the MCGW instance when provisioning and the description should be able to be changed after provisioning.	MCGW-ID-03	When provisioning a MCGW instance the ability to provide a description for the instance should be supported. Changing the description after the instance is provisioned should be supported as well.
			Path - /mcgw/v1/gateways
			• Operation - POST, PATCH
			• Request Body Schema
			  o Object - description
			    ▪ Type - string
			    ▪ Required - true

High	Users should be able to change/upgrade the tier level for the MCGW instance. See pricing section for tier levels.	LDVS-484	Changing/upgrading the tier level of MCGW instance should be supported.
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - mcgw_tier
			    ▪ Type - string
			    ▪ Required - true

High	Users should receive a message when adding a new Fabric Connection to an instance that indicates a tier upgrade is required to add the connection.	LDVS-471	When adding a Fabric Connection to a MCGW instance that would put the total aggregate bandwidth of the instance, a message indicating the attachment failed due to exceeding the tier level should be supported.

	For example: User has a 10G instance with six 1Gbps Fabric Connections attached and tries to add a 5Gbps Fabric Connection, this would put the total aggregate bandwidth of the instance at 11G. Since 11G is over the 10G tier level of the instance an upgrade to the next level would be required.
	Changing/upgrading the tier level of MCGW instance should be supported.
			API response error message - “Error - Attaching the Fabric Connection would exceed the instances current {mcgw_tier} tier level, an upgrade of the instances tier level is required.”
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object - mcgw_tier
			        ▪ Type - string
			        ▪ Required - true

High	Users should receive an identifier for the MCGW instance within the POST response when provisioning a MCGW instance.	LDVS-459	An instance identifier (ID) should be supported for each MCGW instance. Inclusion of the identifier should be supported within the response to a POST request to the following endpoint - /mcgw/v1/gateways
			Each identifier should be randomly generated when an instance is provisioned.
			Path - /mcgw/v1/gateways
			• Operation - POST
			• Request Body Schema
			    o Object - mcgw_id
			        ▪ Type - integer
			        ▪ Read-Only - true

High	Users should be able to reference a MCGW instance using an identifier.	LDVS-473	A parameter for the MCGW instance identifier should be supported in the API endpoint as depicted below. The identifier should be generated once a request is received, and the value should be included within the response.

LUMEN

			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation -GET, PATCH, DEL
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true

High	Users should be able to change the name of a MCGW instance.	LDVS-474	Each MCGW instance should support the ability to have the name changed after being provisioned.
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object - name
			        ▪ Type - string
			        ▪ Required – false

High	Users should be able to delete a MCGW instance.	LDVS-475	Each MCGW instance should support the ability to be deleted after being provisioned.
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - DEL
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true

High	Users should be able to change the description of a MCGW instance.	LDVS-476	Each MCGW instance should support the ability to have the description changed after being provisioned. Path - /mcgw/v1/gateways/{mcgw_id} • Operation - PATCH

			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object -description
			        ▪ Type - string
			        ▪ Required – false

Medium	Users should be able to change the default ASN of a MCGW instance.	LDVS-477	Each MCGW instance should support the ability to have the default ASN changed after being provisioned. This is the local ASN used for each BGP session.
			
			Default ASN = 1
			
			Supported ASN values are:
			• 1 (default)
			• 64512 – 65534 (private)
			• Customer ASN (public)
			
			When a Customer ASN (Public) is used there should be a mechanism to verify the ASN belongs to the customer before the MCGW can utilize the public ASN.
			
			Path - /mcgw/v1/gateways/{mcgw_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object - asn
			        ▪ Type - integer
			        ▪ Required – false

Medium	Users should be able to create both IPv4 and IPv6 BGP peering session on the same MCGW Interface (dual stack)	MCGW-BGP-01	Both IPv4 and IPv6 BGP peering sessions should be supported, with the ability to have both sessions on the same MCGW Interface to support a dual stack configuration.

High	Users should be able to create an IPv4 BGP peering session on each MCGW interface.	LDVS-478	BGP peering should be supported with the ability to create an IPv4 BGP peer per MCGW interface.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/sessions
			
			• Operation - POST
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object - name
			        ▪ Type - string
			        ▪ Required - false
			    o Object- mcgw_interface_id
			        ▪ Type - string
			        ▪ Required - true
			    o Object - address_family
			        ▪ Type - string
			        ▪ Required - false
			        ▪ Enum - “ipv4”,”ipv6”
			        ▪ Default - “ipv4”
			    o Object - local_peer_ip **(future)**
			        ▪ Type - string
			        ▪ Required - true
			    o Object - local_asn **(future)**
			        ▪ Type - integer
			        ▪ Required - false
			    o Object - advertise_default_route
			        ▪ Type - boolean

			▪ Required - false o Object - redistribute_static_routes ▪ Type - boolean ▪ Required - false o Object - redistribute_prefix_list ▪ Type - string ▪ Required - false o Object - inbound_prefix_list ▪ Type - string ▪ Required - false o Object - outbound_prefix_list ▪ Type - string ▪ Required - false o Object - remote_peer_ip ▪ Type - string ▪ Required - true o Object - remote_asn ▪ Type - integer ▪ Required - true o Object - med ▪ Type - integer ▪ Required - false o Object - mulithop_ttl **(future)** ▪ Type - integer ▪ Required - false o Object - md5_authentication ▪ Type - boolean ▪ Required - false o Object - md5_password ▪ Type - string ▪ Required - false

			○ Object - bfd
			    ▪ Type - boolean
			    ▪ Required - false
			○ Object - bfd_interval
			    ▪ Type - integer
			    ▪ Required - false
			    ▪ Enum - "500","1000","1500","2000"
			○ Object - bfd_multiplier
			    ▪ Type - integer
			    ▪ Required - false
			    ▪ Range - 5-10
			
			Association of a prefix-list for each BGP session should be required, both inbound and outbound, before any prefix is allowed to be advertised and received. If a prefix-list is not specified, the default action should be to drop all received prefixes and to not advertise any prefixes.

Medium	Users should be able to create an IPv6 BGP peering session on each MCGW interface.	MCGW-BGP-03	BGP peering should be supported with the ability to create an IPv6 BGP peer per MCGW interface.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions
			
			• Operation - POST
			• Parameter - mcgw_id
			    ○ Type - integer
			    ○ Required - true
			• Parameter - interface_id
			    ○ Type - integer
			    ○ Required - true
			• Request Body Schema

			◦ Object - session_name ▪ Type - string ▪ Required - false ◦ Object - local_peer_ip (future) ▪ Type - string ▪ Required - true ◦ Object - local_asn (future) ▪ Type - integer ▪ Required - false ◦ Object - default_orig ▪ Type - boolean ▪ Required - false ◦ Object - redis_static ▪ Type - boolean ▪ Required - false ◦ Object - redis_prefix_list ▪ Type - integer ▪ Required - false ◦ Object - prefix_list_in ▪ Type - integer ▪ Required - false ◦ Object - prefix_list_out ▪ Type - integer ▪ Required - false ◦ Object - remote_peer_ip ▪ Type - string ▪ Required - true ◦ Object - remote_asn ▪ Type - integer ▪ Required - true ◦ Object - received_as_prepend

			▪ Type - integer ▪ Required - false o Object - advertised_as_prepend ▪ Type - integer ▪ Required - false o Object - local_pref ▪ Type - integer ▪ Required - false o Object - med ▪ Type - integer ▪ Required - false o Object - mulithop_ttl ▪ Type - integer ▪ Required - false o Object - md5_auth ▪ Type - boolean ▪ Required - false o Object - md5_key ▪ Type - string ▪ Required - false o Object - bfd ▪ Type - boolean ▪ Required - false o Object - bfd_interval ▪ Type - integer ▪ Required - false o Object - bfd_multiplier ▪ Type - integer ▪ Required - false Association of a prefix-list for each BGP session should be required, both inbound and outbound, before any prefix is

			allowed to be advertised and received. If a prefix-list is not specified, the default action should be to drop all received prefixes and to not advertise any prefixes.

High	Users should receive a session identifier within the POST response after provisioning an IPv4 BGP peering session.	LDVS-479	A session identifier (ID) should be supported for each IPv4 BGP peering session. Inclusion of the identifier should be supported within the response to a POST request.
			
			Each identifier should be randomly generated when a session is provisioned.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv4/sessions
			
			• Operation - POST
			• Response Body Schema
			    o Object - session_id
			        ▪ Type - integer
			        ▪ Read-Only - true

Medium	Users should receive a session identifier within the POST response after provisioning an IPv6 BGP peering session.	MCGW-BGP-05	A session identifier (ID) should be supported for each IPv6 BGP peering session. Inclusion of the identifier should be supported within the response to a POST request.
			
			Each identifier should be randomly generated when a session is provisioned.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions
			
			• Operation - POST
			• Response Body Schema
			    o Object - session_id
			        ▪ Type - integer

			▪ Read-Only - true

High	Users should be able to modify an existing IPv4 BGP peering session.	[LDVS-480]()	Modification of an existing IPv4 BGP peering session should be supported, with the ability to reference the session by its identifier.
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv4/sessions/{session_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Parameter - session_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - session_name
			    ▪ Type - string
			    ▪ Required - false
			  o Object - local_peer_ip (future)
			    ▪ Type - string
			    ▪ Required - true
			  o Object - local_asn (future)
			    ▪ Type - integer
			    ▪ Required - false
			  o Object - default_orig
			    ▪ Type - boolean
			    ▪ Required - false

			- Object - redis_static - Type - boolean - Required - false - Object - redis_prefix_list - Type - integer - Required - false - Object - prefix_list_in - Type - integer - Required - false - Object - prefix_list_out - Type - integer - Required - false - Object - remote_peer_ip - Type - string - Required - true - Object - remote_asn - Type - integer - Required - true - Object - received_as_prepend - Type - integer - Required - false - Object - advertised_as_prepend - Type - integer - Required - false - Object - local_pref - Type - integer - Required - false - Object - med - Type - integer - Required - false - Object - mulithop_ttl

			▪ Type - integer ▪ Required - false o Object - md5_auth ▪ Type - boolean ▪ Required - false o Object - md5_key ▪ Type - string ▪ Required - false o Object - bfd ▪ Type - boolean ▪ Required - false o Object - bfd_interval ▪ Type - integer ▪ Required - false o Object - bfd_multiplier ▪ Type - integer ▪ Required - false When changes are made to a BGP peering session and multiple BGP peering sessions are configured on the MCGW interface, only the modified session should reset.
Medium	Users should be able to modify an existing IPv6 BGP peering session.	MCGW-BGP-08	Modification of an existing IPv6 BGP peering session should be supported, with the ability to reference the session by its identifier. Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions/{session_id} • Operation - PATCH • Parameter - mcgw_id o Type - integer o Required - true

			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Parameter - session_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - session_name
			    ▪ Type - string
			    ▪ Required - false
			  o Object - local_peer_ip **(future)**
			    ▪ Type - string
			    ▪ Required - true
			  o Object - local_asn **(future)**
			    ▪ Type - integer
			    ▪ Required - false
			  o Object - advertise_default_route
			    ▪ Type - boolean
			    ▪ Required - false
			  o Object - redistribute_static
			    ▪ Type - boolean
			    ▪ Required - false
			  o Object - redistribute_prefix_list
			    ▪ Type - integer
			    ▪ Required - false
			  o Object - inbound_prefix_list
			    ▪ Type - integer
			    ▪ Required - false
			  o Object - outbound_prefix_list
			    ▪ Type - integer
			    ▪ Required - false

- Object - remote_peer_ip
  - Type - string
  - Required - true
- Object - remote_asn
  - Type - integer
  - Required - true
- Object - med
  - Type - integer
  - Required - false
- Object - mulithop_ttl **(future)**
  - Type - integer
  - Required - false
- Object - md5_authentication
  - Type - boolean
  - Required - false
- Object - md5_password
  - Type - string
  - Required - false
- Object - bfd
  - Type - boolean
  - Required - false
- Object - bfd_interval
  - Type - integer
  - Required - false
  - Enum - “500”,”1000”,”1500”,”2000”
- Object - bfd_multiplier
  - Type - integer
  - Required - false

When changes are made to a BGP peering session and multiple BGP peering sessions are configured on the MCGW interface, only the modified session should reset.

High	Users should be required to apply a prefix-list both inbound and outbound before any prefixes are advertised and received. The default filter for all IPv4 and IPv6 BGP peering sessions should be deny all. This is for security and to ensure the user does not generate an outage by flooding routes into their environments.	LDVS-481	By default, no prefixes should be advertised and all received prefixes should be dropped for a new IPv4 and IPv6 BGP peering session. Once a prefix-list is applied inbound and/or outbound, prefixes should then be accepted and advertised based on the rules in the prefix-list.

High	Users should be able to configure MD5 Authentication on an IPv4 BGP peering session.	LDVS-477	MD5 Authentication should be supported on an IPv4 BGP peering session, with the ability to enable and disable MD5 Authentication on each BGP peering session and configuring an authentication key. This should be supported during the initial BGP peering session creation and after.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv4/sessions
			
			• Operation - POST
			• Parameter - mcgw_id
			    o Type - integer
			    o Required - true
			• Parameter - interface_id
			    o Type - integer
			    o Required - true
			• Request Body Schema
			    o Object - md5_auth
			        ▪ Type - boolean (default is false)
			        ▪ Required - false
			    o Object - md5_key

			▪ Type - string
			▪ Required - false
			
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv4/sessions/{session_id}
			
			• Operation - PATCH
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Parameter - session_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - md5_auth
			    ▪ Type - boolean (default is false)
			    ▪ Required - false
			  o Object - md5_key
			    ▪ Type - string
			    ▪ Required - false

Medium	Users should be able to configure MD5 Authentication on an IPv6 BGP peering session.	MCGW-BGP-11	MD5 Authentication should be supported on an IPv6 BGP peering session, with the ability to enable and disable MD5 Authentication on each BGP peering session and configuring an authentication key. This should be supported during the initial BGP peering session creation and after.

			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions
			• Operation - POST
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - md5_auth
			    ▪ Type - boolean (default is false)
			    ▪ Required - false
			  o Object - md5_key
			    ▪ Type - string
			    ▪ Required - false
			Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions/{session_id}
			• Operation - PATCH
			• Parameter - mcgw_id
			  o Type - integer
			  o Required - true
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Parameter - session_id
			  o Type - integer
			  o Required - true

			- Request Body Schema - Object - md5_auth - Type - boolean (default is false) - Required - false - Object - md5_key - Type - string - Required - false
Medium	Users should be able to change the MCGW local ASN used for an IPv4 BGP peering session on a per MCGW Interface basis. This should not result in changing the MCGW instance ASN.	MCGW-BGP-12	Changing of the MCGW local ASN used for an IPv4 BGP peering session should be supported on a per MCGW Interface basis. Supported ASN values are: 1 (default) 64512 – 65534 (private) - Customer ASN (public) Changing of the local ASN on an existing session should require a PUT to allow the session to be updated and reset. Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv4/sessions/{session_id} - Operation - PUT - Parameter - mcgw_id - Type - integer - Required - true - Parameter - interface_id - Type - integer - Required - true - Parameter - session_id - Type - integer

			◦ Required - true • Request Body Schema ◦ Object - local_asn ▪ Type - integer ▪ Required - true When a Customer ASN (Public) is used there should be a mechanism to verify the ASN belongs to the customer before the MCGW can utilize the public ASN. Local ASN must be the same for all BGP peering sessions configured on a MCGW interface. Changing the local_asn requires modifying the entire BGP peering session configuration, resulting in a teardown of the current session and turn up of a new session.
Medium	Users should be able to change the MCGW local ASN used for an IPv6 BGP peering session on a per MCGW Interface basis. This should not result in changing the MCGW instance ASN.	MCGW-BGP-13	Changing of the MCGW local ASN used for an IPv6 BGP peering session should be supported on a per MCGW Interface basis. Supported ASN values are: • 1 (default) • 64512 – 65534 (private) • Customer ASN (public) Changing of the local ASN on an existing session should require a PUT to allow the session to be updated and reset. Path - /mcgw/v1/gateways/{mcgw_id}/bgp/{interface_id}/ipv6/sessions/{session_id} • Operation - PUT • Parameter - mcgw_id

LUMEN

			- Type - integer - Required - true - Parameter - interface_id - Type - integer - Required - true - Parameter - session_id - Type - integer - Required - true - Request Body Schema - Object - local_asn - Type - integer - Required - true When a Customer ASN (Public) is used there should be a mechanism to verify the ASN belongs to the customer before the MCGW can utilize the public ASN. Local ASN must be the same for all BGP peering sessions configured on a MCGW interface. Changing the local_asn requires modifying the entire BGP peering session configuration, resulting in a teardown of the current session and turn up of a new session.
High	Users should be able to configure a default BGP MED (Multiple Exit Discriminator) value for all advertised prefixes for a BGP peering sessions.	LDVS-481	Default BGP MED for all advertised prefixes for a BGP peering session should be supported, resulting in all advertised prefixes to assigned the MED value. - Request Body Schema - Object - med - Type - integer - Required - false Allowed med values are 0 to 4294967295

High	Users should be able to configure a default BGP Local Preference for all received prefixes for a BGP peering session.	LDVS-483	Default BGP Local Preference for all received prefixes for a BGP peer session should be supported, resulting in all received prefixes to be assigned the default local preference value. • Request Body Schema ○ Object - local_pref ▪ Type - integer ▪ Required - false Allowed local_pref values are integers from 0 to 4294967295
Medium	Users should be able to configure EBGP Multihop per BGP peering session.	MCGW-BGP-16	EBGP Multihop should be supported with the ability to enable EBGP Multihop per BGP peering session by indicating the number of hops allowed for the BGP peering session. • Request Body Schema ○ Object - mulithop_ttl ▪ Type - integer ▪ Required - false Allowed multihop_ttl values are integers from 2 to 5
High	Users should be able to configure a default BGP AS path prepend for all received prefixes for a BGP peering session.	LDVS-470	Default BGP AS path prepend for all received prefixes for a BGP peering session should be supported, resulting in all received prefixes to have their AS path prepended with the default AS path prepend value. • Request Body Schema ○ Object - received_as_prepend ▪ Type - integer ▪ Required - false Allowed received_as_prepend values are 1 to 3

High	Users should be able to configure a default BGP AS path prepend for all advertised prefixes for a BGP peering session.	LDVS-485	Default BGP AS path prepend for all advertised prefixes for a BGP peering session should be supported, resulting in all advertised prefixes to have their AS path prepended with the default AS path prepend value.
			• Request Body Schema
			  ○ Object - advertised_as_prepend
			    ▪ Type - integer
			    ▪ Required - false
			Allowed advertised_as_prepend values are 1 to 3

Medium	Users should be able to configure BGP communities that can be tied to a prefix-list for matching and associated to specific BGP peers to set a community attribute on an advertised prefix. Not fully defined.	MCGW-BGP-18	BGP communities should be supported outbound, multiple BGP communities should be able to be created, with the ability to tie a prefix-list to the community as matching criteria, and the ability to set a community attribute for each community. Any community configured should be able to be applied to any BGP peer to set a community for an advertised prefix. Object - community (string)

High	Users should be able to configure default information originate on a BGP peering session to allow a default route (0.0.0.0/0) to be advertised to the BGP neighbor.	LDVS-486	Default information originate should be supported on a BGP peering session.
			• Request Body Schema
			  ○ Object - default_orig
			    ▪ Type - boolean
			    ▪ Required - false
			Default value for default_orig should be false.

Medium	Users should be able to redistribute all static routes into BGP if the static routes	MCGW-BGP-11	Redistribution of all static routes (see MCGW-Static-02) into BGP should be supported if the static routes are associated to the same MCGW Interface.

	are associated to the same MCGW Interface.		• Request Body Schema
			  ○ Object - redis_static
			    ▪ Type - boolean
			    ▪ Required - false
			Default value for redis_static should be false
			For a static route to be redistributed the prefix for each route must be in the outbound prefix-list associated to the BGP peering session.

High	Users should be able to enable or disable Bidirectional Forwarding Detection (BFD) on a BGP peering session.	MCGW-BGP-12	Bidirectional Forwarding Detection (BFD) should be supported for each BGP peering session, with the ability to enable or disable for each BGP session.
			• Request Body Schema
			  ○ Object - bfd
			    ▪ Type - boolean
			    ▪ Required - false
			Default value for BFD should be false.

High	Users should be able to configure the Bidirectional Forwarding Detection (BFD) interval on a BGP peering session.	LDVS-487	Bidirectional Forwarding Detection (BFD) interval should be supported for each BGP peering session, with the ability to configure the interval individually for each BGP peering session.
			• Request Body Schema
			  ○ Object - bfd_interval
			    ▪ Type - integer
			    ▪ Required - false
			Default value for bfd_interval should be 500ms.
			Supported values should be 500 to 2000ms, with 500 increments. 500, 1000, 1500, 2000

High	Users should be able to configure the Bidirectional Forwarding Detection (BFD) multiplier on a BGP peering session.	LDVS-488	Bidirectional Forwarding Detection (BFD) multiplier should be supported for each BGP peering session, with the ability to configure the interval individually for each BGP session.
			• Request Body Schema
			  ○ Object - bfd_multiplier
			    ▪ Type - integer
			    ▪ Required - false
			Default value for bfd_multiplier should be 5.
			Supported values should be 5-10.

High	Users should be able to view BGP route details.	LDVS-489	Visibility into BGP route details should be supported.
			Path - /mcgw/v1/gateways/{mcgw_id}/routes
			• Operation - GET
			• Parameter - mcgw_id
			  ○ Type - integer
			  ○ Required - true
			• Request Body Schema
			  ○ Object - address_family
			    ▪ Type - string
			    ▪ Required - false
			    ▪ Enum - "ipv4","ipv6","all"
			    ▪ Default - ipv4
			The following data should be returned for each route:
			• Prefix
			• Mask
			• Next hop IP address
			• Autonomous System (AS) path
			• Community tags
			• Local Preference

			• MED • Administrative distance • Metric

High	Users should be able to create one or more IPv4 prefix-lists.	LDVS-491	Creation of IPv4 prefix-lists should be supported with the ability to use a prefix-list across multiple MCGW instances. The prefix-list should be a global resource within a customer's account.
			
			Path - /mcgw/v1/prefix-list/ipv4/lists
			
			• Operation - POST
			• Request Body Schema
			    ○ Object - prefix_list_id
			        ▪ Type - integer
			        ▪ Required - true
			    ○ Object - prefix_list_name
			        ▪ Type - string
			        ▪ Required -false
			    ○ Object - prefixes
			        ▪ Type - array
			        ▪ Required - true
			            • Object - prefix_id
			                ○ Type - integer
			                ○ Required - true
			            • Object - prefix
			                ○ Type - string
			                ○ Required - true
			            • Object - prefix-length
			                ○ Type - string
			                ○ Required - true
			            • Object - match_type
			                ○ Type - string

			○ Required - false • Object - local_pref ○ Type - Integer ○ Required - false • Object - med ○ Type - integer ○ Required - false • Object - as_prepend ○ Type - integer ○ Required - false • Object - prefix_tag ○ Type - string ○ Required - false
Medium	Users should be able to create one or more IPv6 prefix-lists.	MCGW-PLST-01	Creation of IPv6 prefix-lists should be supported with the ability to use a prefix-list across multiple MCGW instances. The prefix-list should be a global resource within a customer's account. Path - /mcgw/v1/prefix-list/ipv6/lists • Operation - POST • Request Body Schema ○ Object - prefix_list_id ▪ Type - integer ▪ Required - true ○ Object - prefix_list_name ▪ Type - string ▪ Required -false ○ Object - prefixes ▪ Type - array ▪ Required - true • Object - prefix_id

			○ Type - integer ○ Required - true • Object - prefix ○ Type - string ○ Required - true • Object - prefix-length ○ Type - string ○ Required - true • Object - match_type ○ Type - string ○ Required - false • Object - local_pref ○ Type - Integer ○ Required - false • Object - med ○ Type - integer ○ Required - false • Object - as_prepend ○ Type - integer ○ Required - false • Object - prefix_tag ○ Type - string ○ Required - false
High	Users should be able to set an identifier for a prefix-list.	LDVS-490	A prefix-list identifier should be supported with the ability to configure the value. This identifier should be an integer and required to be set when the prefix-list is initially created. • Request Body Schema ○ Object - prefix_list_id ▪ Type - integer ▪ Required - true

			• Values for the identifier should be 1 - 250

High	Users should be able to modify a specific prefix-list using its assigned identifier.	LDVS-492	Modification of an existing prefix-list should be supported, with the ability to reference the prefix-list by an assigned identifier during initial creation.
			Path - /mcgw/v1/prefix-list/ipv4/lists/{prefix_list_id}
			• Operation - PATCH
			• Parameter - prefix_list_id
			    o Type - integer
			    o Required - true
			Path - /mcgw/v1/prefix-list/ipv6/lists/{prefix_list_id}
			• Operation - PATCH
			• Parameter - prefix_list_id
			    o Type - integer
			    o Required - true

High	Users should be able to delete prefixes from a prefix-list.	LDVS-493	Deletion of specific prefixes from a prefix-list should be supported.
			Path - /mcgw/v1/prefix-list/ipv4/lists/{prefix_list_id}
			• Operation - PATCH
			• Parameter - prefix_list_id
			    o Type - integer
			    o Required - true
			Path - /mcgw/v1/prefix-list/ipv6/lists/{prefix_list_id}
			• Operation - PATCH
			• Parameter - prefix_list_id
			    o Type - integer
			    o Required - true

High	Users should be able to indicate the type of match (exact or longer) to perform on a prefix.	LDVS-494	Match type should be supported on a prefix in a prefix-list.
			• Request Body Schema
			  ○ Object - match_type
			    ▪ Type - string
			    ▪ Required - false
			Values for match_type should be:
			• Exact (default)
			• Longer

High	User should be able to set the local preference of a prefix within a prefix-list.	LDVS-495	Local preference values should be supported for each prefix with a prefix-list.
			• Request Body Schema
			  ○ Object - local_pref
			    ▪ Type - Integer
			    ▪ Required - false

High	Users should be able to set a tag for each prefix within a prefix-list.	LDVS-496	Tage values should be supported on a per prefix basis within a prefix-list.
			• Request Body Schema
			  ○ Object - prefixes
			    ▪ Type - array
			    ▪ Required - true
			      • Object - prefix_id
			        ○ Type - integer
			        ○ Required - true
			      • Object - prefix_tag
			        ○ Type - string
			        ○ Required - false

High	Users should be able to view IPv4 prefix-lists.	LDVS-497	Viewing of IPv4 prefix-list should be supported. Path - /mcgw/v1/prefix-list/ipv4/lists • Operation - GET • Response Body Schema o Object - prefix_list_name ▪ Type - string o Object - prefixes ▪ Type - array • Object - prefix_id o Type - integer • Object - prefix o Type - string • Object - prefix-length o Type - string • Object - match_type o Type - string • Object - local_pref o Type - Integer • Object - med o Type - integer • Object - as_prepend o Type - integer • Object - prefix_tag o Type - string
High	Users should be able to view an IPv4 prefix-list by referencing the ID of the list.	LDVS-498	Viewing of an IPv4 prefix-list by the prefix_list_id should be supported. Path - /mcgw/v1/prefix-list/ipv4/lists/{prefix_list_id} • Operation - GET • Parameter - prefix_list_id

			○ Type - integer ○ Required -true • Response Body Schema ○ Object - prefix_list_name ▪ Type - string ○ Object - prefixes ▪ Type - array • Object - prefix_id ○ Type - integer • Object - prefix ○ Type - string • Object - prefix_length ○ Type - string • Object - match_type ○ Type - string • Object - local_pref ○ Type - Integer • Object - med ○ Type - integer • Object - as_prepend ○ Type - integer • Object - prefix_tag ○ Type - string
Medium	Users should be able to view IPv6 prefix-lists.	MCGW-PLST-09	Viewing of IPv6 prefix-list should be supported. Path - /mcgw/v1/prefix-list/ipv6/lists • Operation - GET • Response Body Schema ○ Object - prefix_list_name ▪ Type - string ○ Object - prefixes

<table>
<tr>
<td></td>
<td></td>
<td></td>
<td>
Type - array

Object - prefix_id
Type - integer
Object - prefix
Type - string
Object - prefix-length
Type - string
Object - match_type
Type - string
Object - local_pref
Type - Integer
Object - med
Type - integer
Object - as_prepend
Type - integer
Object - prefix_tag
Type - string

</td>
</tr>
<tr>
<td>Medium</td>
<td>Users should be able to view an IPv6 prefix-list by referencing the ID of the list.</td>
<td>MCGW-PLST-10</td>
<td>
Viewing of an IPv6 prefix-list by the prefix_list_id should be supported.
Path - /mcgw/v1/prefix-list/ipv6/lists/{prefix_list_id}

Operation - GET
Parameter - prefix_list_id
Type - integer
Required -true
Response Body Schema

Object - prefix_list_name
Type - string
Object - prefixes
Type - array

</td>
</tr>
</table>

			• Object - prefix_id
			    ○ Type - integer
			• Object - prefix
			    ○ Type - string
			• Object - prefix-length
			    ○ Type - string
			• Object - match_type
			    ○ Type - string
			• Object - local_pref
			    ○ Type - Integer
			• Object - med
			    ○ Type - integer
			• Object - as_prepend
			    ○ Type - integer
			• Object - prefix_tag
			    ○ Type - string

High	Users should be able to configure IPv4 static routes on any connection/interface instead of using BGP.	LDVS-499	Static IPv4 routes should be supported with the ability to add and remove static routes per connection/interface. Static routes should be able to be used in place of BGP, thus not making BGP configuration mandatory for any connection/interface.
			
			When a route is created a unique identifier should be associated to each individual route.
			
			• Object - route_id
			    ○ Type - integer
			    ○ Required - true
			
			Path - /mcgw/v1/gateways/{mcgw_id}/static-routes
			
			• Operation - POST, PATCH, GET, DEL
			• Request Body Schema

			○ Object - routes
			  ▪ Type - array
			  ▪ Required - true
			    • Object - prefix
			      ○ Type - string
			      ○ Required - true
			    • Object - prefix_length
			      ○ Type - string
			      ○ Required - true
			    • Object - dst_address
			      ○ Type - string
			      ○ Required - true
			    • Object - distance
			      ○ Type - string
			      ○ Required - false
			    • Object - tag
			      ○ Type - string
			      ○ Required - false

Medium	Users should be able to configure IPv6 static routes on any connection/interface instead of using BGP.	MCGW-Static-02	Static IPv6 routes should be supported with the ability to add and remove static routes per connection/interface. Static routes should be able to be used in place of BGP, thus not making BGP configuration mandatory for any connection/interface.
			When a route is created a unique identifier should be associated to each individual route.
			• Object - route_id
			  ○ Type - integer
			  ○ Required - true
			Path - /mcgw/v1/gateways/{mcgw_id}/static/ipv6/routes

			• Operation - POST, PATCH, GET, DEL
			• Request Body Schema
			  ○ Object - routes
			    ▪ Type - array
			    ▪ Required - true
			      • Object - prefix
			        ○ Type - string
			        ○ Required - true
			      • Object - prefix_length
			        ○ Type - string
			        ○ Required - true
			      • Object - dst_address
			        ○ Type - string
			        ○ Required - true
			      • Object - distance
			        ○ Type - string
			        ○ Required - false
			      • Object - tag
			        ○ Type - string
			        ○ Required - false

High	Users should be able to delete static IPv4 routes using its assigned identifier.	[LDVS-500]()	Deletion of IPv4 static routes should be supported with the ability to reference the routes to remove by the identifier tied to each route.
			Path - /mcgw/v1/gateways/{mcgw_id}/static/ipv4/routes
			• Operation - DEL
			• Request Body Schema
			  ○ Object - routes
			    ▪ Type - array
			    ▪ Required - true
			      • Object - route_id

			○ Type - integer ○ Required - true
Medium	Users should be able to delete static IPv6 routes using its assigned identifier.	MCGW-Static-04	Deletion of IPv6 static routes should be supported with the ability to reference the routes to remove by the identifier tied to each route. Path - /mcgw/v1/gateways/{mcgw_id}/static/ipv6/routes • Operation - DEL • Request Body Schema ○ Object - routes ▪ Type - array ▪ Required - true • Object - route_id ○ Type - integer ○ Required - true
High	~~Users should be able to view all IPv4 static routes.~~	~~LDVS-501~~	~~Visibility into all provisioned IPv4 static routes should be supported. The response to a request should be an array.~~ ~~Path - /mcgw/v1/gateways/{mcgw_id}/static/ipv4/routes~~ • ~~Operation - GET~~ • ~~Response Body Schema~~ ○ ~~Object - routes~~ ▪ ~~Type - array~~ ▪ ~~Required - true~~ • ~~Object - route_id~~ ○ ~~Type - string~~ ○ ~~Required - true~~ • ~~Object - prefix~~ ○ ~~Type - string~~ ○ ~~Required - true~~

			• ~~Object - prefix_length~~
			    ○ ~~Type - string~~
			    ○ ~~Required - true~~
			• ~~Object - dst_address~~
			    ○ ~~Type - string~~
			    ○ ~~Required - true~~
			• ~~Object - distance~~
			    ○ ~~Type - string~~
			    ○ ~~Required - true~~
			• ~~Object - tag~~
			    ○ ~~Type - string~~
			    ○ ~~Required - true~~

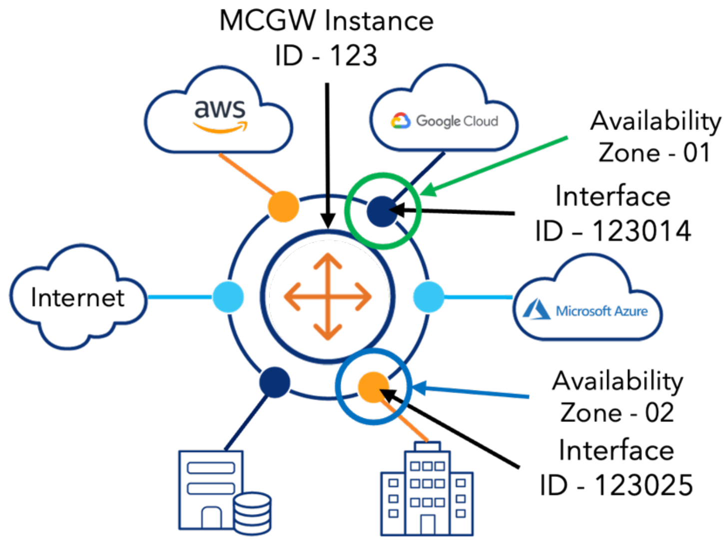
High	Users should be able to view all IPv4 and IPv6 static routes.	LDVS-501	Visibility into all provisioned IPv4 static routes should be supported. The response to a request should be an array.
			Path - /mcgw/v1/gateways/{mcgw_id}/static/routes
			• Operation - GET
			• Request Body Schema
			    ○ Object - address_family
			        ▪ Type - string
			        ▪ Required - false
			        ▪ Enum - “ipv4”,”ipv6”,”all”
			        ▪ Default - ipv4
			• Response Body Schema
			    ○ Object - routes
			        ▪ Type - array
			        ▪ Required - true
			            • Object - route_id
			                ○ Type - string
			                ○ Required - true
			            • Object - prefix

			○ Type - string ○ Required - true • Object - prefix_length ○ Type - string ○ Required - true • Object - dst_address ○ Type - string ○ Required - true • Object - distance ○ Type - string ○ Required - true • Object - tag ○ Type - string ○ Required - true
Medum	~~Users should be able to view all IPv6 static routes~~	~~MCGW-Static-06~~	~~Visibility into all provisioned IPv6 static routes should be supported. The response to a request should be an array.~~ ~~Path - /mcgw/v1/gateways/{mcgw_id}/static/ipv6/routes~~ • ~~Operation - GET~~ • ~~Response Body Schema~~ ○ ~~Object - routes~~ ▪ ~~Type - array~~ ▪ ~~Required - true~~ • ~~Object - route_id~~ ○ ~~Type - string~~ ○ ~~Required - true~~ • ~~Object - prefix~~ ○ ~~Type - string~~ ○ ~~Required - true~~

			• ~~Object - prefix_length~~
			  ○ ~~Type - string~~
			  ○ ~~Required - true~~
			• ~~Object - dst_address~~
			  ○ ~~Type - string~~
			  ○ ~~Required - true~~
			• ~~Object - distance~~
			  ○ ~~Type - string~~
			  ○ ~~Required - true~~
			• ~~Object - tag~~
			  ○ ~~Type - string~~
			  ○ ~~Required - true~~

High	There should be a construct/concept of a MCGW Interface	LDVS-502	A MCGW Interface construct/concept should be supported for the attachment and termination of a Fabric Connection and Fabric Port.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			
			• Operation - POST, PATCH, GET, DEL
			• Request Body Schema
			  ○ Object - interface_id
			    ▪ Type - integer
			    ▪ Required - true
			  ○ Object - admin_state
			    ▪ Type - string
			    ▪ Enum - “enabled”, “disabled”
			    ▪ Required - false
			    ▪ Default - enabled
			  ○ Object - name
			    ▪ Type - string
			    ▪ Required - false
			  ○ Object - description

			▪ Type - string ▪ Required - false o Object - bandwidth ▪ Type - integer (number of bits/sec) ▪ Required - false o Object - ipv4_address ▪ Type - object • Object - address ○ Type - string ○ Required - true • Object - prefix_length ○ Type - string ○ Required - true o Object - ipv6_address ▪ Type - object • Object - address ○ Type - string ○ Required - false • Object - prefix_length ○ Type - string ○ Required - false
High	Each MCGW Interface should have a unique identifier that will be used to identify the interface.	LDVS-503	Unique interface identifiers (UID) should be supported to distinguish each MCGW Interface. For example: Identifier could be comprised of the MCGW instance ID, the availability zone selected, and a sequential integer that represents the MCGW Interface ID.

			 The identifier should be used as the parameter to identify an interface within an API call. Path - /mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id} • Operation - GET, PATCH, PUT, DEL • Parameter - interface_id 　o Type - integer 　o Required - true
High	Users should be billed for each MCGW Interface.	[LDVS-465]()	MCGW Interface should support a billing construct.
High	Users should not be billed for a MCGW Interface until a Fabric Connection is attached to the interface.	[LDVS-464]()	Billing for a MCGW Interface should only start once a Fabric Connection is attached to the MCGW Interface.

LUMEN

High	Users should be able to have MCGW Interfaces that do not have a Fabric Connection attached to them.	LDVS-466	The ability to have a MCGW Interface provisioned that does not have a Fabric Connection attached to it should be supported.
High	MCGW Interface identifiers should not change after a user has provisioned a MCGW Interface.	LDVS-467	Once a MCGW Interface has been provisioned and assigned a unique identifier that unique identifier associated to the MCGW Interface should never change.
High	MCGW Interfaces should have multiple states.	LDVS-468	Multiple states of a MCGW interface should be supported. States should be determined based on the status of the Layer2 connection (Fabric Connection) and Layer 3 configuration. State should be represented as {Layer 2}/{Layer 3}, where Layer 2 and/or Layer 3 can be either Up or Down. Supported states of a MCGW interface: • Down/Down • Up/Down • Up/Up • Admin Down Layer 2 state should be indicated as Down under the following conditions: • Fabric Connection or Fabric Port is not attached to the MCGW interface • Fabric Connection or Fabric Port attached to the MCGW interface is in a disabled state • Fabric Port attached to the MCGW interface is in the up/down state • Fabric Port attached to the MCGW interface is in the down/down state

			Layer 2 state should be indicated as Up under the following conditions: • Fabric connection is attached to the MCGW interface, and the Fabric Connection is in the enabled state • Fabric Port attached to the MCGW interface is in the up/up state Layer 3 state should be indicated as Down under the following conditions: • IP addresses have not been assigned to the MCGW interface • IP addresses have been assigned to the MCGW interface, but the Layer 2 state is Down Layer 3 state should be indicated as Up under the following condition: • IP addresses have been assigned to the MCGW interface and Layer 2 state is Up Admin Down should be indicated under the following conditions, regardless of Layer 2 and 3 states: • After instantiation of an interface (default state should be disabled) • After a user has disabled the interface When a MCGW interface is disabled (Admin Down) all Layer 3 connectivity/reachability should cease. Enabling and/or disabling of a MCGW interface should not result in a change to the attached Fabric Connection or Fabric Port when either is attached.

High	Users should be able to provision MCGW Interfaces.	LDVS-469	Provisioning of new MCGW interfaces should be supported.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST
			• Response Body Schema
			  o Object - interface_id
			    ▪ Type - integer
			    ▪ Required - true

High	Users should be able to delete MCGW Interfaces.	LDVS-504	Deletion of a MCGW Interface should be supported.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - DEL
			• Response Body Schema
			  o Object - interface_id
			    ▪ Type - integer
			    ▪ Required - true

High	Users should be able to enable/disable a MCGW Interface, thus changing the admin state of the interface.	LDVS-505	Changing the admin state of a MCGW Interface should be supported.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - admin_state
			    ▪ Type - string
			    ▪ Required - true
			    ▪ Enum: “enable” “disable”

High	Users should be able to configure a name for each MCGW Interface, at time of and after provisioning.	LDVS-506	Configuration of a name for each MCGW Interface should be supported, when provisioned and any time after.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST, PATCH
			• Request Body Schema
			  o Object - name
			    ▪ Type - string
			    ▪ Required - false
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - name
			    ▪ Type - string
			    ▪ Required - false

High	Users should be able to configure a description for each MCGW Interface, at time of and after provisioning.	LDVS-507	Configuration of a description for each MCGW Interface should be supported, when provisioned and any time after.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST
			• Request Body Schema
			  o Object - description
			    ▪ Type - string
			    ▪ Required - false

			Path -
			/mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - description
			    ▪ Type - string
			    ▪ Required - false

High	A MCGW Interface should inherit the bandwidth of the attached Fabric Connection.	LDVS-508	Inheritance of a Fabric Connection’s bandwidth should be supported. When a Fabric Connection is attached to a MCGW Interface, its bandwidth value should be inherited.

Medium	Users should be able to configure the bandwidth of a MCGW Interface.
	Note: If the total aggregate bandwidth of an instance is calculated by totaling the bandwidth value of all active MCGW Interfaces, allowing a user to change this value will result in an incorrect value for total aggregate bandwidth.
		MCGW-INT-13	Configuration of a bandwidth value for the MCGW Interface should be supported. Until a Fabric Connection is attached to a MCGW Interface the bandwidth should not be modifiable.
			Path -
			/mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Parameter - interface_id
			  o Type - integer
			  o Required - true
			• Request Body Schema
			  o Object - bandwidth
			    ▪ Type - integer (bits/sec)
			    ▪ Required - false

High	User should be able to use any IPv4 address space, i.e. they should be able to bring their own IP addresses.	LDVS-509	Any IPv4 address from any IP space with any prefix-length (netmask) should be supported.

High	Users should be able to configure an IPv4 address and subnet mask for each MCGW Interface, at time of and after provisioning.	LDVS-510	Configuration of an IPv4 address and subnet mask should be supported, when provisioned and any time after.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST
			• Request Body Schema
			  o Object - ipv4_address
			    ▪ Object - address
			      • Type - string
			      • Required - true
			    ▪ Object - prefix_length
			      • Type - string
			      • Required - true
			Path - /mcgw/v1/ gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Request Body Schema
			  o Object - ipv4_address
			    ▪ Object - address
			      • Type - string
			      • Required - true
			    ▪ Object - prefix_length
			      • Type - string
			      • Required - true

Medium	User should be able to use any IPv6 address space, i.e. they should be able to bring their own IP addresses.	MCGW-INT-16	Any IPv6 address from any IP space with any prefix-length should be supported.

Medium	Users should be able to configure an IPv6 address and subnet mask for each MCGW Interface, at time of and after provisioning.	MCGW-INT-17	Configuration of an IPv6 address and subnet mask should be supported, when provisioned and any time after.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST
			• Request Body Schema
			  o Object - ipv6_address
			    ▪ Type - object
			      • Object - address
			        o Type - string
			        o Required - true
			      • Object - prefix_length
			        o Type - string
			        o Required - true
			Path - /mcgw/v1/interfaces/{interface_id}
			• Operation - PATCH
			• Request Body Schema
			  o Object - ipv6_address
			    ▪ Type - object
			      • Object - address
			        o Type - string
			        o Required - true
			      • Object - prefix_length
			        o Type - string
			        o Required - true

Medium	User should be able to configure both IPv4 and IPv6 on the same interface, allowing for a dual stack interface.	MCGW-INT-18	Configuration of both IPv4 and IPv6 on the same interface should be supported.

High	Users should be to configure the admin state of each MCGW Interface, at time of and after provisioning.	LDVS-511	Configuration of the admin state of a MCGW Interface should be supported, when provisioned and any time after.
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces
			• Operation - POST
			• Request Body Schema
			  o Object - admin_state
			    ▪ Type - string
			    ▪ Enum - “Enabled”, “Disabled”
			    ▪ Required - false
			Path - /mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}
			• Operation - PATCH
			• Request Body Schema
			  o Object - admin_state
			    ▪ Type - string
			    ▪ Enum - “Enabled”, “Disabled”
			    ▪ Required - false

High	Users should be able to view the current telemetry data for a MCGW Interface. The data should be collected/calculated on a 3-minute interval, the response should include the most recent collection values, but also allow a user to indicate a predefined time period.	LDVS-512	Visibility of MCGW Interface telemetry data should be supported. Telemetry data can be sourced from VLAN interface on the PE if such an interface exists and/or the Fabric Connection attached to the MCGW Interface. Data collection/calculation every 3 minutes should be supported, with the response including data from the last

<table>
<tr>
<td></td>
<td>The following telemetry data should be supported:

• Receive Utilization Percentage
• Transmit Utilization Percentage
• Receive bits per second (bps)
• Transmit bits per second (bps)
• Receive Packets per Second (pps)
• Transmit Packets per Second (pps)
• Minimum Receive bits per second (bps) - time period
• Maximum Receive bits per second (bps) - time period
• Minimum Transmit bits per second (bps) - time period
• Maximum Transmit bits per second (bps) - time period
• Minimum Receive Packets per Second (pps) - time period
• Maximum Receive Packets per Second (pps) - time period
• Minimum Transmit Packets per Second (pps) - time period
• Maximum Transmit Packets per Second (pps) - time period</td>
<td></td>
<td>interval (most recent), unless a predefined time period is provided in the request.

Path - /telemetry/mcgw/v1/gateways/{mcgw_id}/interfaces/{interface_id}

• Operation - GET
• Parameter - interface_id
    o Type - string
    o Required - true
• Request Body Schema
    o Object - time_period
        ▪ Type - string
        ▪ Enum - “15mins”, “30mins”, “45mins”, “60mins”
        ▪ Required - false

Response:

• Response Body Schema
    o Object - last_interval
        ▪ Type - string
        ▪ Format - “2025-05-23 00:00:00”
        ▪ Timezone - UTC???
    o Object - name
        ▪ Type - string
    o Object - description
        ▪ Type - string
    o Object - state
        ▪ Type - string
        ▪ Enum: “Down/Down” “Up/Down” “Up/Up” “Admin Down"
    o Object - current</td>
</tr>
</table>

			▪ Type - array • Object - rx_utilization ○ Type - integer • Object - tx_utilization ○ Type - integer • Object - rx_bps ○ Type - integer • Object - tx_bps ○ Type - integer • Object - rx_pps ○ Type - integer • Object - tx_pps ○ Type - integer ○ Object - time_period ▪ Type - array • Object - min_rx_bps ○ Type - integer • Object - max_rx_bps ○ Type - integer • Object - min_tx_bps ○ Type - integer • Object - max_tx_bps ○ Type - integer • Object - min_rx_pps ○ Type - integer • Object - max_rx_pps ○ Type - integer • Object - min_tx_pps ○ Type - integer • Object - max_tx_pps

			○ Type - integer
High	Users should be able to detach a Fabric Connection from a MCGW Interface.	[LDVS-513]()	Detaching a Fabric Connection from a MCGW interface should be supported, with the ability to leave the interface with no Fabric Connection attached. Performed through the Fabric Connection endpoint, the Fabric Connection endpoint should support an object that is utilized to confirm the Fabric Connection should be detached from the MCGW Interface. See Fabric Connections PRD

Medium	User should be able to attach a Fabric Port to a MCGW Interface.
	
	See Fabric Connections PRD - [FC-PRD]()
		MCGW-INT-22	Attachment of a Fabric Port to a MCGW interface should be supported by provisioning a Transparent Point-to-Point Fabric Connection (EPL).
			
			Below is the Fabric Connections endpoint for a Transparent P2P
			
			Path - /fabric/v1/connections/transparent/p2p
			• Operation - POST
			• Request Body Schema
			  ○ Object - interface_id
			    ▪ Type - string
			    ▪ Required - true

High	User should be able to attach an M-UNI to a MCGW Interface See Fabric Connections PRD		Attachment of an M-UNI to a MCGW interface should be supported by provisioning a point-to-point fabric connection (EPL) This provides a back-up for the timeline to develop the compatibility of Fabric Port

High	Users should be able to upgrade the MCGW instance tier level when attaching a Fabric Connection to a MCGW Interface. See Fabric Connections PRD - FC-PRD	LDVS-514	Upgrading of a MCGW instance's tier level when attaching a Fabric Connection to a MCGW Interface should be supported. Path - /fabric/v1/connections/.* ○ Object - mcgw ▪ Type - object • Object - mcgw_id ○ Type - integer ○ Required - true • Object - interface_id ○ Type - integer ○ Required - true • Object - mcgw_tier ○ Type - string ○ Required - false
Medium	Users should be able to create subinterfaces when attaching a Dedicated Cloud Fabric Connection, each subinterface should map to the Vlan tag the user created within their cloud environment.	MCGW-INT-24	Provisioning of subinterfaces should be supported when attaching a Dedicated Cloud Fabric Connection. Need to build out the details.
High	Users should be able to provision a new Virtual Point-to-Point Fabric Connection that can be attached to an existing or new MCGW Interface. See Fabric Connections PRD - FC-PRD	LDVS-515	Provisioning of a new Virtual Point-to-Point Fabric Connection (EVPL) should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface. Below is the Fabric Connections endpoint for a Virtual P2P Path - /fabric/v1/connections/virtual/p2p • Operation - POST • Request Body Schema ○ Object - interface_id

LUMEN

			▪ Type - string ▪ Required – true
Medium	Users should be able to provision a new Transparent Point-to-Point Fabric Connection that can be attached to an existing or new MCGW Interface. See Fabric Connections PRD - FC-PRD		Provisioning of a new Transparent Point-to-Point Fabric Connection (EPL) should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface. Below is the Fabric Connections endpoint for a Transparent P2P Path - /fabric/v1/connections/transparent/p2p • Operation - POST • Request Body Schema o Object - interface_id ▪ Type - string ▪ Required - true
High	Users should be able to provision an AWS Hosted Cloud Fabric Connection that can be attached to an existing or new MCGW Interface. See Fabric Connections PRD - FC-PRD	LDVS-516	Provisioning of a new AWS Hosted Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface. Below is the Fabric Connections endpoint for an AWS Hosted Cloud Path - /fabric/v1/connections/cloud/hosted/aws • Operation - POST • Request Body Schema o Object - interface_id ▪ Type - string ▪ Required - true

High	Users should be able to provision an GCP Hosted Cloud Fabric Connection that can be attached to an existing or new MCGW Interface.
	
	See Fabric Connections PRD - FC-PRD
		LDVS-517	Provisioning of a new GCP Hosted Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface.
			
			Below is the Fabric Connections endpoint for an GCP Hosted Cloud
			
			Path - /fabric/v1/connections/cloud/hosted/gcp
			
			• Operation - POST
			• Request Body Schema
			    ○ Object - interface_id
			        ▪ Type – string
			        ▪ Required - true

High	Users should be able to provision an Azure Hosted Cloud Fabric Connection that can be attached to an existing or new MCGW Interface.
	
	See Fabric Connections PRD - FC-PRD
	
	Azure will require two MCGW interfaces, one for the primary EVC and another for the secondary EVC, requiring two requests, one for each EVC.
		LDVS-518	Provisioning of a new Azure Hosted Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface.
			
			Below is the Fabric Connections endpoint for an Azure Hosted Cloud
			
			Path - /fabric/v1/connections/cloud/hosted/azure
			
			• Operation - POST
			• Request Body Schema
			    ○ Object - mcgw
			        ▪ Type - object
			            • Object - mcgw_id
			                ○ Type - integer
			                ○ Required - true
			            • Object - interface_id
			                ○ Type - integer
			                ○ Required - true
			            • Object - mcgw_tier

			○ Type - string ○ Required - false
High	Users should be able to provision an Oracle Hosted Cloud Fabric Connection that can be attached to an existing or new MCGW Interface. See Fabric Connections PRD - [FC-PRD]()	[LDVS-519]()	Provisioning of a new Oracle Hosted Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface. Below is the Fabric Connections endpoint for an Oracle Hosted Cloud Path - /fabric/v1/connections/cloud/hosted/oracle • Operation - POST • Request Body Schema ○ Object - interface_id ▪ Type - string ▪ Required - true
Medium	Users should be able to provision an AWS Dedicated Cloud Fabric Connection that can be attached to an existing or new MCGW Interface. See Fabric Connections PRD - [FC-PRD]()	MCGW-FC-06	Provisioning of a new AWS Dedicated Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface. Below is the Fabric Connections endpoint for an AWS Dedicated Cloud Path - /fabric/v1/connections/cloud/dedicated/aws • Operation - POST • Request Body Schema ○ Object - interface_id ▪ Type – string ▪ Required - true
Medium	Users should be able to provision a GCP Dedicated Cloud Fabric Connection that	MCGW-FC-07	Provisioning of a new GCP Dedicated Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface.

	can be attached to an existing or new MCGW Interface.
	
	See Fabric Connections PRD - FC-PRD
			Below is the Fabric Connections endpoint for an GCP Dedicated Cloud
			
			Path - /fabric/v1/connections/cloud/dedicated/gcp
			• Operation - POST
			• Request Body Schema
			  ○ Object - interface_id
			    ▪ Type – string
			    ▪ Required - true

Medium	Users should be able to provision an Azure Dedicated Cloud Fabric Connection that can be attached to an existing or new MCGW Interface.
	
	See Fabric Connections PRD - FC-PRD
		MCGW-FC-08	Provisioning of a new Azure Dedicated Cloud Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface.
			
			Below is the Fabric Connections endpoint for an Azure Dedicated Cloud
			
			Path - /fabric/v1/connections/cloud/dedicated/azure
			• Operation - POST
			• Request Body Schema
			  ○ Object - interface_id
			    ▪ Type - string
			    ▪ Required - true

Low	Users should be able to attach an existing Fabric Connection to a new or existing MCGW Interface.	MCGW-FC-09	Attaching an existing Fabric Connection should be supported with the ability to attach the Fabric Connection to a new or existing MCGW interface.
High	Users should be able to scale bandwidth up and down.	LDVS-520	Increasing and decreasing the bandwidth of an attached Fabric Connection should be supported.
Medium	Users should be able to attach an existing VRF from their IP VPN to a MCGW Interface.	MCGW-MPLS-01	Attachment of an existing IP VPN VRF to a MCGW interface should be supported.

LUMEN

Medium	Users should be able to detach a VRF from their IP VPN, from a MCGW Interface.	MCGW-MPLS-02	Detaching an IP VPN VRF from a MCGW interface should be supported.
Medium	Users should be able to attach multiple IP VPN VRFs to a MCGW.	MCGW-MPLS-03	Attachment of multiple IP VPN VRFs to a MCGW should be supported. Each VRF should be attached to separate MCGW interfaces, multiple VRFs on a single MCGW interface should not be supported.
Medium	Users should be able to utilize Availability Zones for redundancy. See Fabric Connections PRD - FC-PRD	MCGW-HA-01	Multiple Availability Zones should be supported, where each zone represents a different AS3549 PE router (VAR).

High	Users should be able to view a single global route table view.	LDVS-521	Combined route table view should be supported. This view should consist of all routes within the instances global VRF.
			
			Path - /mcgw/v1/gateways/{mcgw_id}/route-table
			
			• Operation - GET
			• Parameter - mcgw_id
			    ○ Type - integer
			    ○ Required - true
			• Request Body Schema
			    ○ Object - route
			        ▪ Type - string
			        ▪ Required - false

High	User should be able to see the following information for each route in the table: • **Destination Network:** The IP address and subnet mask of a network the device needs to reach.	LDVS-522	Each route in the route table should support the following fields. • **Destination Network** • **Next Hop IP Address** • **Interface** • **Administrative Distance and Metric** • **Route Source**

<table>
<tr>
<td></td>
<td>

Next Hop IP Address: The IP address of the next router in the path to the destination network.
Interface: The interface on the device through which packets are sent to reach the next hop.
Administrative Distance (AD) and Metric: Indicates the reliability of the route and the cost of using that route.
Route Source: Indicates how the route was learned (e.g., connected, static, BGP).
Route Timestamp: Identifies how much time has passed since the route was learned.

</td>
<td></td>
<td>

Route Timestamp

Path - /mcgw/v1/gateways/{mcgw_id}/route-table

Operation - GET
Parameter - mcgw_id

Type - integer
Required - true

Request Body Schema

Object - route

Type - array
Required - false

Object - dest_network
Type - string
Required - false

Object - next_hop_ip
Type - string
Required - false

Object - interface_id
Type - integer
Required - false

Object - admin_metric
Type - string
Required - false

Object - route_source
Type - string
Required - false

Object - route_timestamp
Type - string
Required - false

</td>
</tr>
</table>

High	Users should be able to search for a specific route within a route table.	LDVS-522	Route table searches should be supported with the ability to provide a route to search for within a route table.
			Path - /mcgw/v1/gateways/{mcgw_id}/route-table/{route}
			• Operation - GET
			• Parameter - mcgw_id
			  ○ Type - integer
			  ○ Required - true
			• Parameter - route
			  ○ Type - string
			  ○ Required - true

High	Users should be able to search for routes based on source interface, which interface the routes were received on.	LDVS-522	Route table searches should be supported with the ability to provide a source interface for the routes.
			Path - /mcgw/v1/gateways/{mcgw_id}/route-table/{interface_id}
			• Operation - GET
			• Parameter - mcgw_id
			  ○ Type - integer
			  ○ Required - true
			• Parameter - interface_id
			  ○ Type - string
			  ○ Required - true

High	Users should be able to search for routes based on route type.	MCGW-RTBL-05	Route table searches should be supported with the ability to provide the route type. Path - /mcgw/v1/gateways/{mcgw_id}/route-table/{route_type} • Operation - GET • Parameter - mcgw_id ○ Type - integer

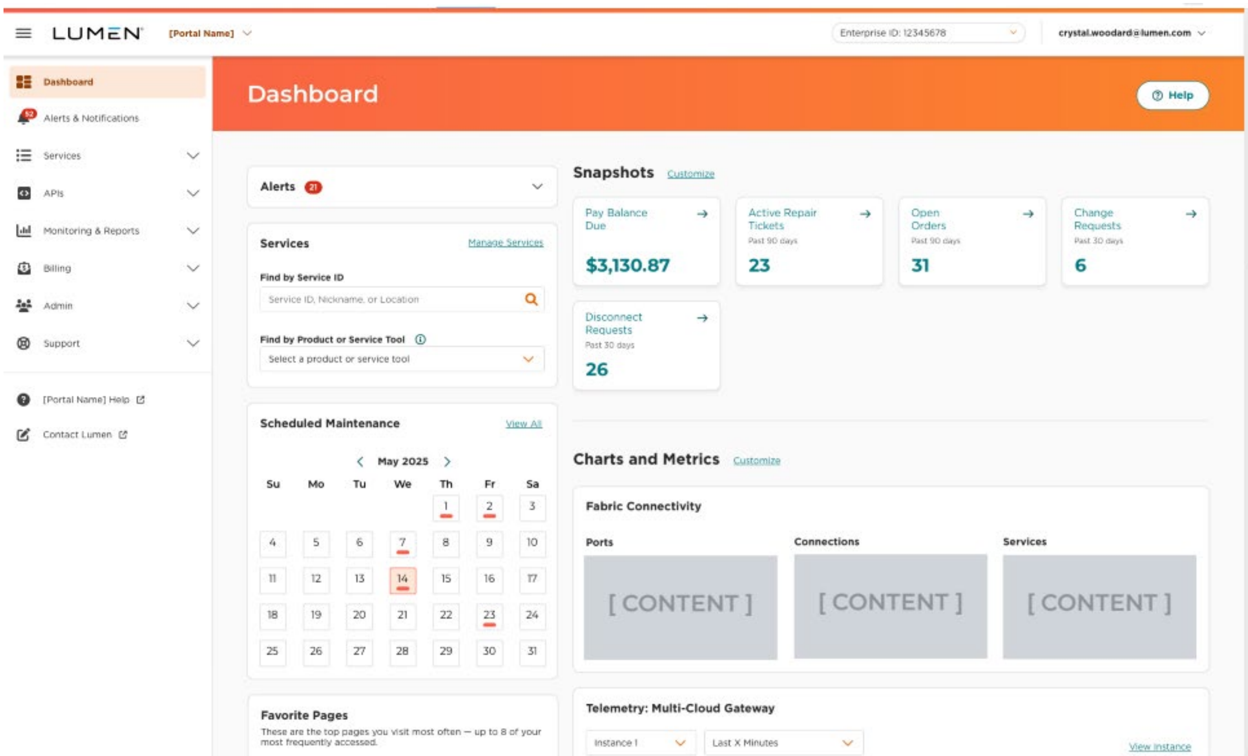
			○ Required - true • Parameter - route_type ○ Type - string ○ Required - true Supported route type values: • bgp • static • connected
Low	User should be able to interconnect their MCGW with another customers MCGW. This is for support of a future Extranet-as-a-Service product.	MCGW-EaaS-01	Interconnecting one or more MCGWs should be supported with the ability to interconnect a customer's MCGW with another customer's MCGW.
Low	User should be able to interconnect their MCGW with a providers MCGW. This is for support of a future Cloud Exchange service.	MCGW-CX-01	Interconnecting one or more MCGWs should be supported with the ability to interconnect a customer's MCGW with a provider's MCGW.
Medium	User should be able to provision a Fabric Connection to an Internet OnRamp (AS3356), to provide Internet access through the MCGW.	MCGW-IOD-01	Provisioning of a Fabric Connection from a MCGW Interface to an Internet OnRamp should be supported.
Medium	User should be able to select the Internet OnRamp they want to provision a Fabric Connection to provide Internet access through the MCGW.	MCGW-IOD-02	Selection of an Internet OnRamp should be supported when provisioning a Fabric Connection to the Internet. Internet OnRamp is an Internet peering point on AS 3356, which are typically tied to a metro.

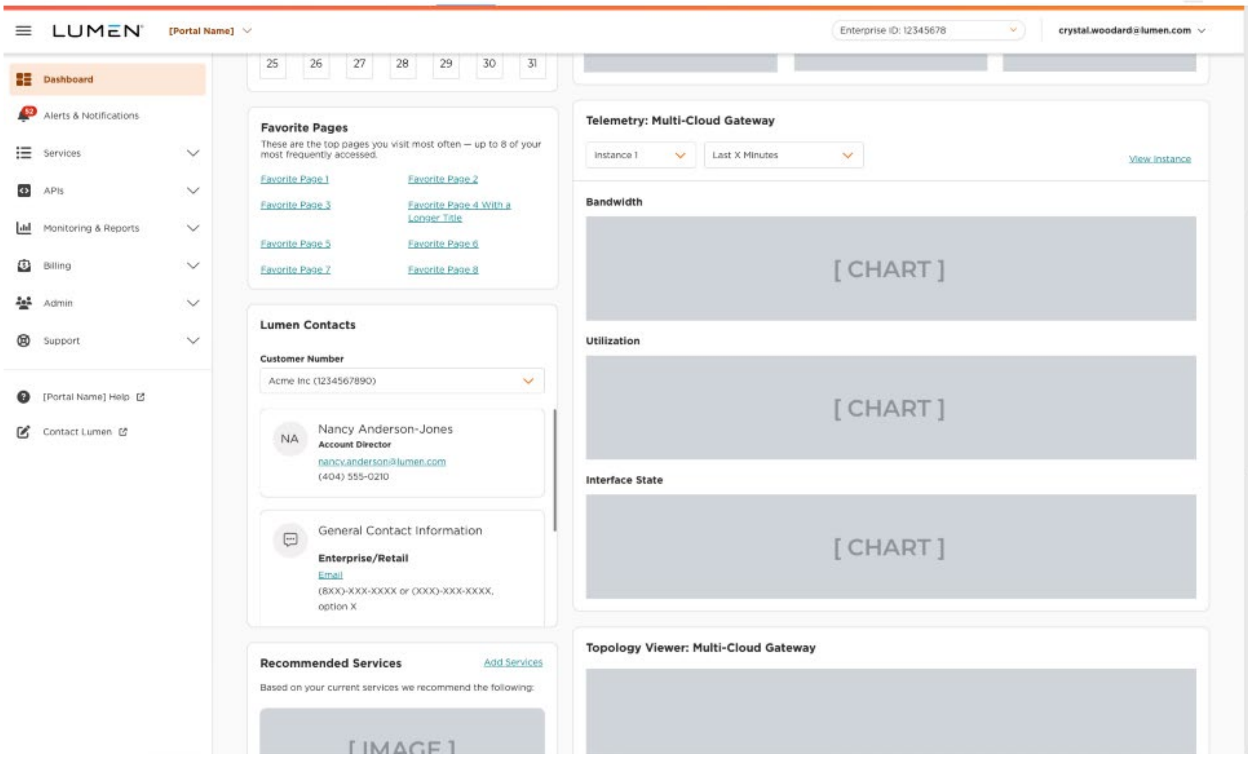
4.5 Customer Experience (CX/UX) (PM)

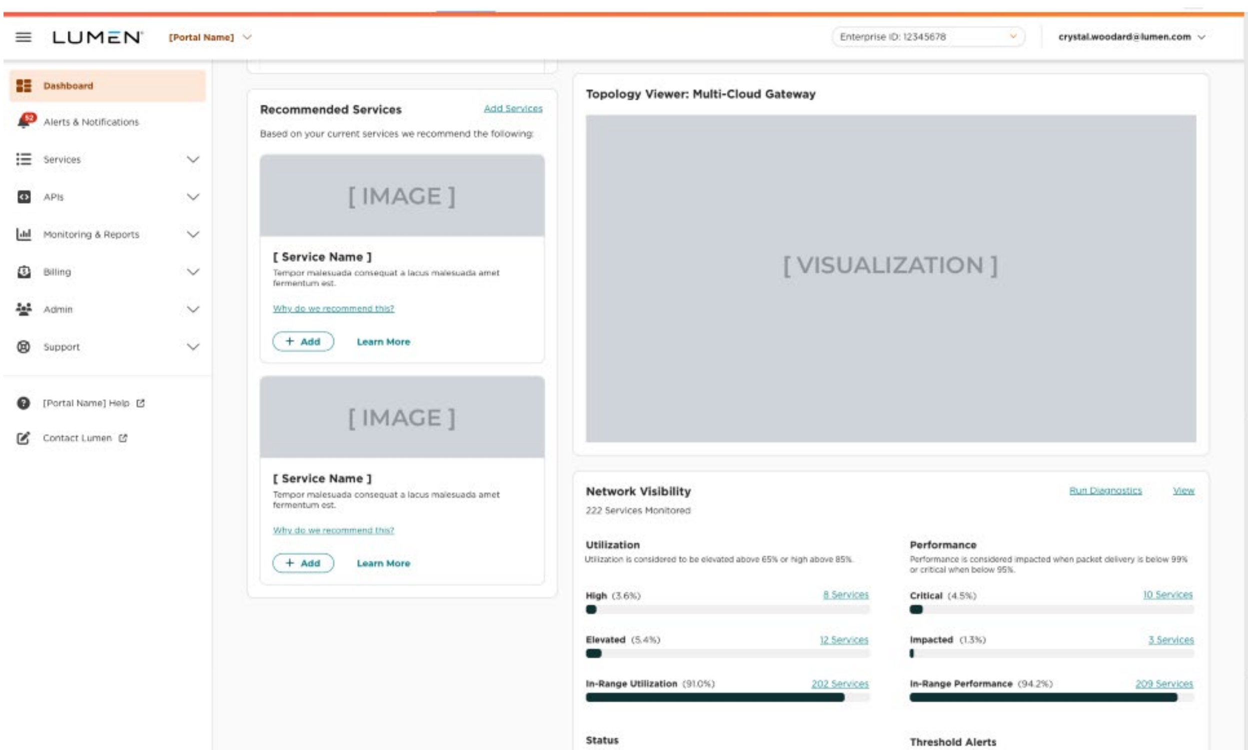
The following are the details of each customer and user experiences for each of the three Go-to-Market (GTM) motions from multiple personas perspective.

4.5.1 Platform-Led

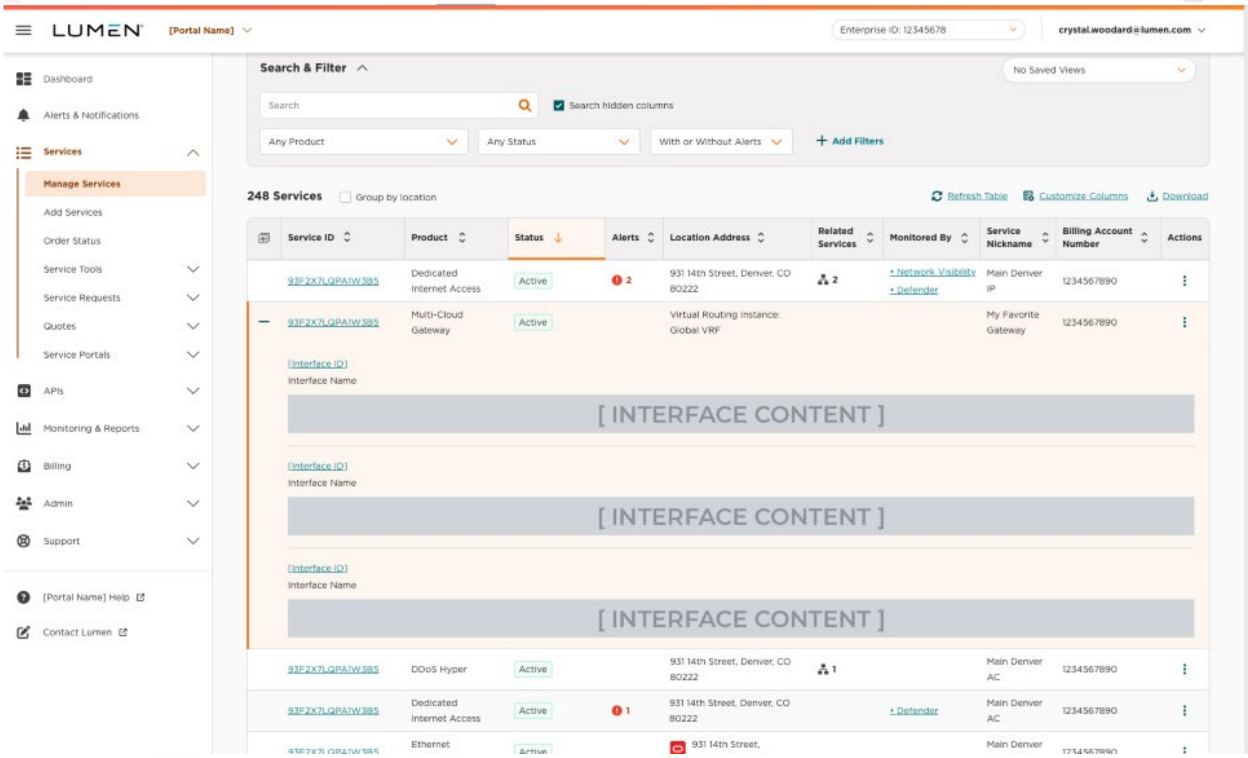
Dashboard




LUMEN
[Portal Name]
Enterprise ID: 12345678
crystal.woodard@lumen.com
Dashboard
Alerts & Notifications
Services
APIs
Monitoring & Reports
Billing
Admin
Support
[Portal Name] Help
Contact Lumen
25 26 27 28 29 30 31
Favorite Pages
These are the top pages you visit most often — up to 8 of your most frequently accessed.
Favorite Page 1
Favorite Page 2
Favorite Page 3
Favorite Page 4 With a Longer Title
Favorite Page 5
Favorite Page 6
Favorite Page 7
Favorite Page 8
Lumen Contacts
Customer Number
Acme Inc (1234567890)
NA
Nancy Anderson-Jones
Account Director
nancy.anderson@lumen.com
(404) 555-0210
General Contact Information
Enterprise/Retail
Email
(8XX)-XXX-XXXX or (XXX)-XXX-XXXX, option X
Recommended Services
Add Services
Based on your current services we recommend the following:
Telemetry: Multi-Cloud Gateway
Instance 1
Last X Minutes
View Instance
Bandwidth
[CHART]
Utilization
[CHART]
Interface State
[CHART]
Topology Viewer: Multi-Cloud Gateway


LUMEN
[Portal Name]
Enterprise ID: 12345678
crystal.woodard@lumen.com
Dashboard
Alerts & Notifications
Services
APIs
Monitoring & Reports
Billing
Admin
Support
[Portal Name] Help
Contact Lumen
Recommended Services
Add Services
Based on your current services we recommend the following:
[IMAGE]
[Service Name]
Tempor malesuada consequat a lacus malesuada amet fermentum est.
Why do we recommend this?
+ Add
Learn More
[IMAGE]
[Service Name]
Tempor malesuada consequat a lacus malesuada amet fermentum est.
Why do we recommend this?
+ Add
Learn More
Topology Viewer: Multi-Cloud Gateway
[VISUALIZATION]
Network Visibility
Run Diagnostics
View
222 Services Monitored
Utilization
Utilization is considered to be elevated above 65% or high above 85%.
High (3.6%)
8 Services
Elevated (5.4%)
12 Services
In-Range Utilization (91.0%)
202 Services
Performance
Performance is considered impacted when packet delivery is below 99% or critical when below 95%.
Critical (4.5%)
10 Services
Impacted (1.3%)
3 Services
In-Range Performance (94.2%)
209 Services
Status
Review the status of your monitored locations and services.
Threshold Alerts

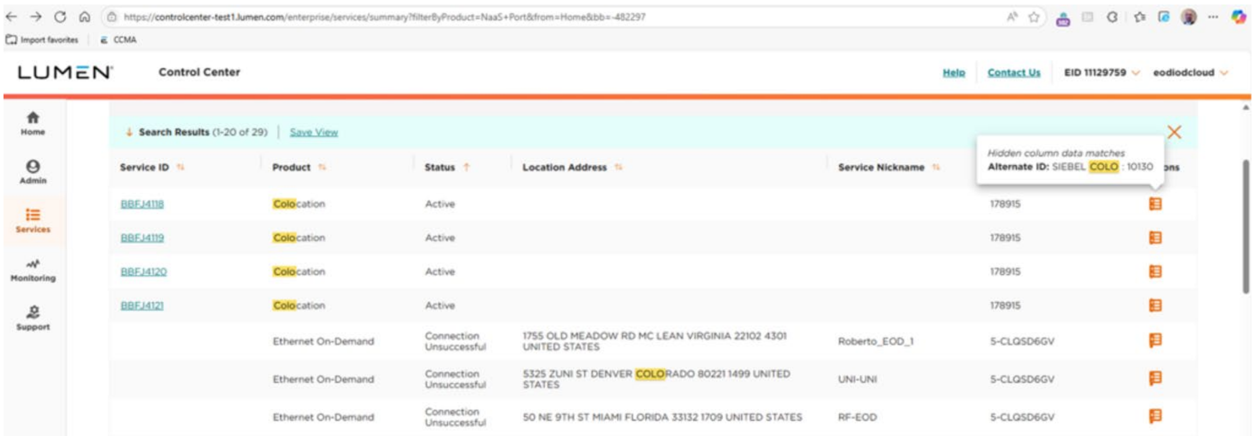
Manage Services



The Manage Services screen will have the ability to expand the view of each MCGW Instance in order to reveal the inventory of MCGW Interfaces which are attached to it (as in a Parent – Child relationship) - coming in a future release (tech not available for MVP 1). The MCGW Instance will utilize the standard column headers as designed in CC for use with all products with the understanding the MCGW Instance data will need to be mapped to existing data fields appropriately. In the event there is a data point that does not map logically to the existing column header, a new data column should be created and entered into the list of available column names which are selectable by the user.

A new column will be created for MCGW Interfaces. It will be selected for display by users using the Customize Columns feature.

If a user searches for a term that matches data for an Interface, but the user has not chosen to display that column, the result will be indicated using the “hidden column data match” feature which will be released in Control Center in the next release.



When the child interfaces are expanded into view, the following data will be displayed

- Interface Name
- Nickname
- IP Address
- Connection Type (“virtual pt-pt", “hosted cloud connection”)
- Status (up/up, down/down, etc)

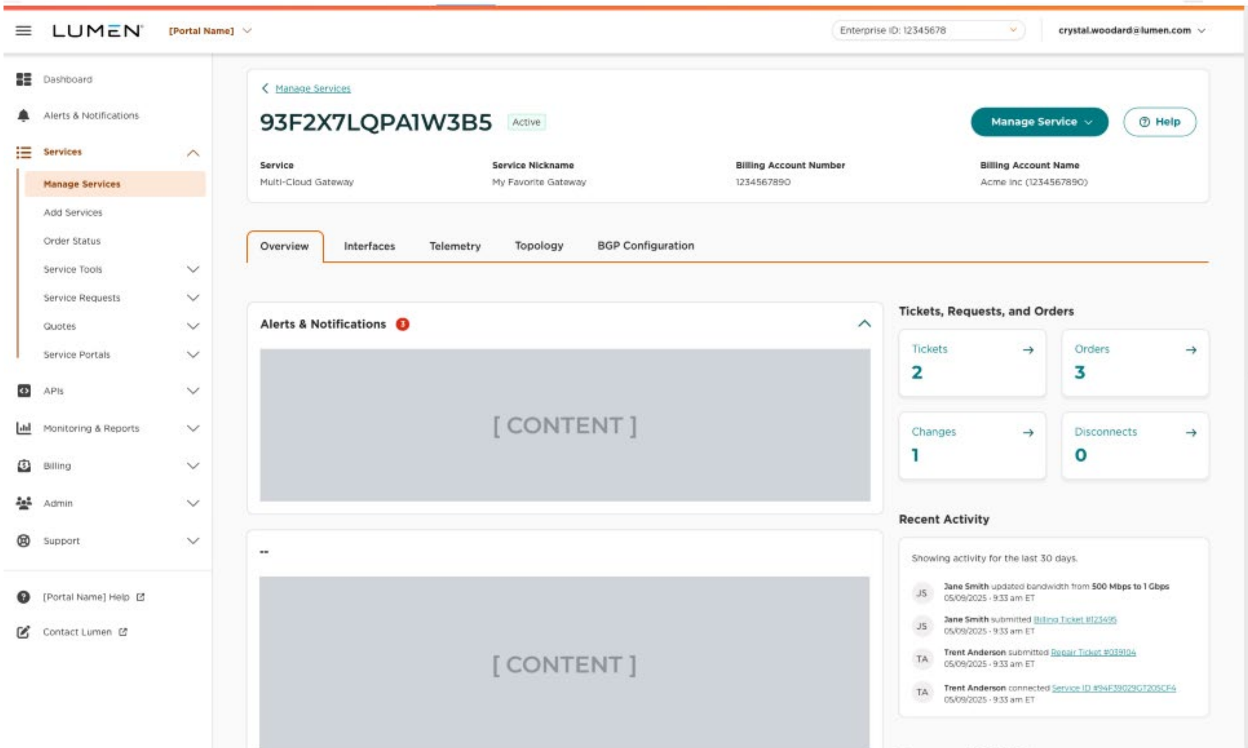
The action burger/kabob will contain the following actions for MCGW Instances

- Service Details (hyperlink)
- Update Nickname
- Add Interface
- Modify Service

The MCGW Interface, when displayed as a child, will have the following action options in the burger/kabob:

- Service Details
- Update Nickname
- Add Interface
- Modify Service

Service Details - Overview



Tickets, Requests and Orders

- There are no orders for MCGW, that section can be removed
- Tickets should be specific to MCGW requests
- Changes and Disconnects are instantiated on demand, those sections can be removed

The Overview page will contain the following tabs

Overview

Details

Interfaces

Telemetry

BGP Configuration

Routing

Overview Tab

This tab will contain the topology as it is the most critical set of foundational information the customer will want to see (what instances and interfaces do I have instantiated right now)

Other relevant data: ?

Details Tab

This tab will contain the following data/widgets:

Overall service details of the MCGW Interface

Alerts and Notifications

Recent Activity

This content section will contain a summary of activities on Instance and related products/services for last 30 days.

Service Details

- Let's hide "Existing VRF" data point until we roll out ability to attach an existing vrf in a future phase
- Service ID and Circuit ID would be assigned to products like Fabric Port, not MCGW Instance or interface. Those data points should appear in the Related Services Content area

Telemetry Tab

This tab will contain the following data in reference to MCGW performance

- Receive Utilization Percentage
- Transmit Utilization Percentage
- Receive bits per second (bps)
- Transmit bits per second (bps)
- Receive Packets per Second (pps)
- Transmit Packets per Second (pps)
- Minimum Receive bits per second (bps) - time period
- Maximum Receive bits per second (bps) - time period
- Minimum Transmit bits per second (bps) - time period
- Maximum Transmit bits per second (bps) - time period
- Minimum Receive Packets per Second (pps) - time period
- Maximum Receive Packets per Second (pps) - time period
- Minimum Transmit Packets per Second (pps) - time period
- Maximum Transmit Packets per Second (pps) - time period

Routing Tab

This tab contains sections for the following data

- BGP Configurations
- Prefix lists
- Full routing table

Billing Details

Related Services

This is an inventory view of all the related products and services which are being used in collaboration with the MCGW Interface.

MCGW Interfaces

Fabric Connections

Fabric Ports

Future Reference: Managed Services, Security services, etc

The data displayed for each inventory item will include:

Service ID

Product Type (e.g. Fabric Connection)

Status (tbd: up/up, down/up, or green/red circles as indicators?)

Routes Tab

This view should consist of all routes within the instances global VRF. MUST be able to view all BGP and/or static route details associated with a customer's multi-cloud gateway instance.

The following data should be returned for each route in the table:

IMPORTANT Note: Internal routing details MUST NOT be displayed to the customer, for security reasons. This includes internal IPs, router names, network-facing interface labels, full global route tables, or any signaling information like BGP AS paths or community tags that might hint at internal routing policies.

- Destination Prefix (IP/mask) - Show only customer's own network (e.g., their assigned IP blocks or VLANs
- Next hop IP address - Next hop should only be displayed if it points to a customer-facing handoff, i.e. internal router IPs must NOT be displayed, thus protecting internal infrastructure.
- Next hop Alias - The next hop represented as a "MCGW Interface Alias" where the prefix was advertised from.
- MCGW Interface Name – If the interface is logically assigned to the customer and reveals nothing about the internal network design.
- Last Updated – To show when the route prefix was changed (learned/flap) on the MCGW.
- Last Refresh – To show when the route prefix was last updated.
- MED - The Multi-Exit Discriminator (MED) value for the prefix, used to influence routing decisions.
- Local Preference - The local preference value for the route, used to influence routing decisions within the AS.
- AS Path Prepend - The AS path for the prefix, indicating the sequence of ASes through which the route has passed.

- Route Type – Display whether the route is BGP, static, or connected.

See example below:

+-----------------------+---------------------+----------------+----------------------+--------------------+-----------
---------+-----+-----------------+-----------------+------------+

| Dest Prefix | Next hop IP addr | Next hop Alias | MCGW Interface Name | Last Updated | Last Refresh | MED | Local Pref | AS Path | Route Type |

+-----------------------+---------------------+----------------+----------------------+--------------------+-----------
---------+-----+-----------------+-----------------+------------+

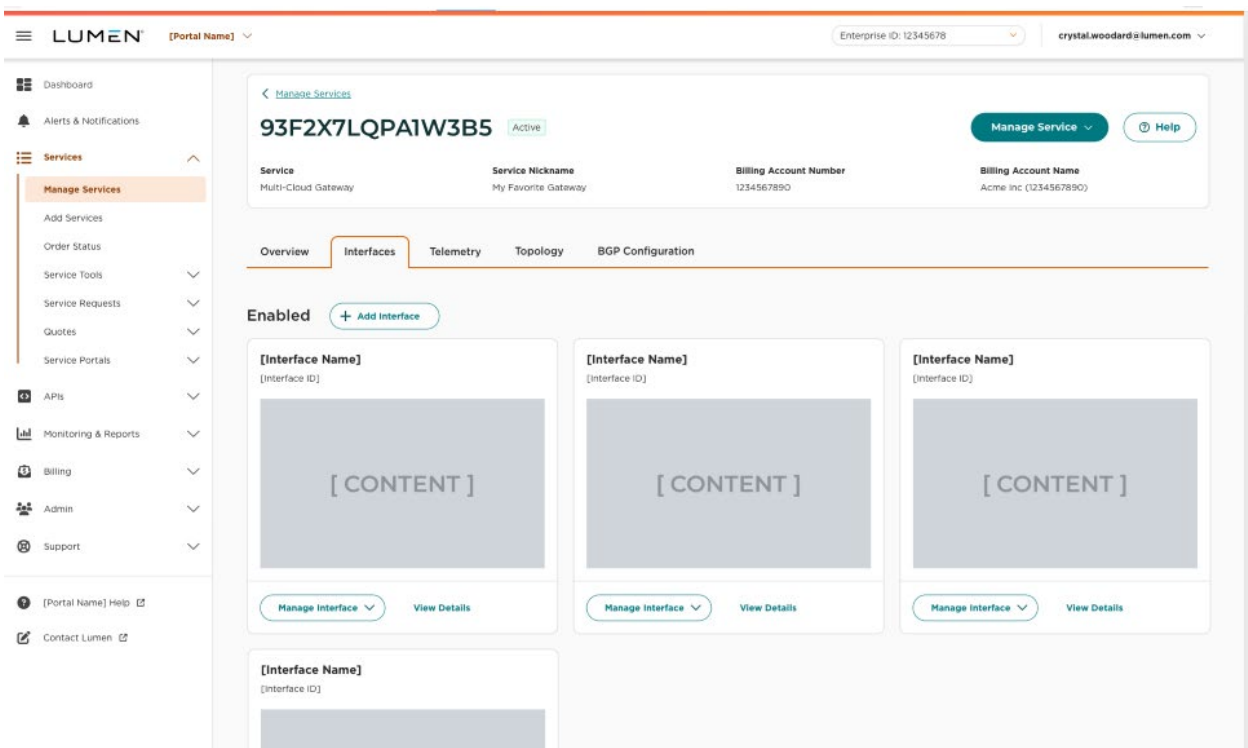
| 192.168.10.0/24 | 203.0.113.1 | MCGW-Alias-1 | Customer-Intf-1 | 2023-10-01 12:30:00 | 2023-10-01 12:45:00 | 100 | 200 | 65001 65002 | BGP |

| 10.10.20.0/24 | 198.51.100.2 | MCGW-Alias-2 | Customer-Intf-2 | 2023-10-02 08:15:00 | 2023-10-02 08:30:00 | 150 | 100 | 65003 65004 | Static |

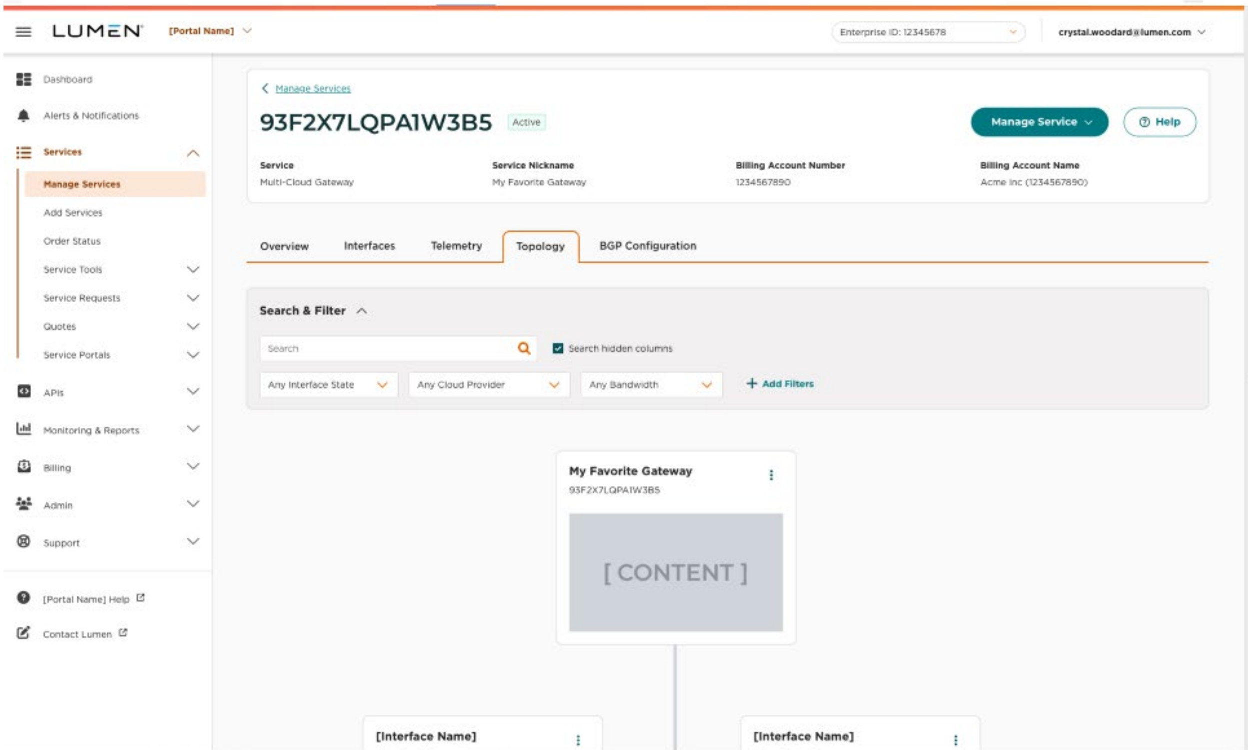
| 172.16.30.0/24 | | MCGW-Alias-3 | Customer-Intf-3 | 2023-10-03 09:00:00 | 2023-10-03 09:15:00 | 200 | 150 | 65005 | Connected |

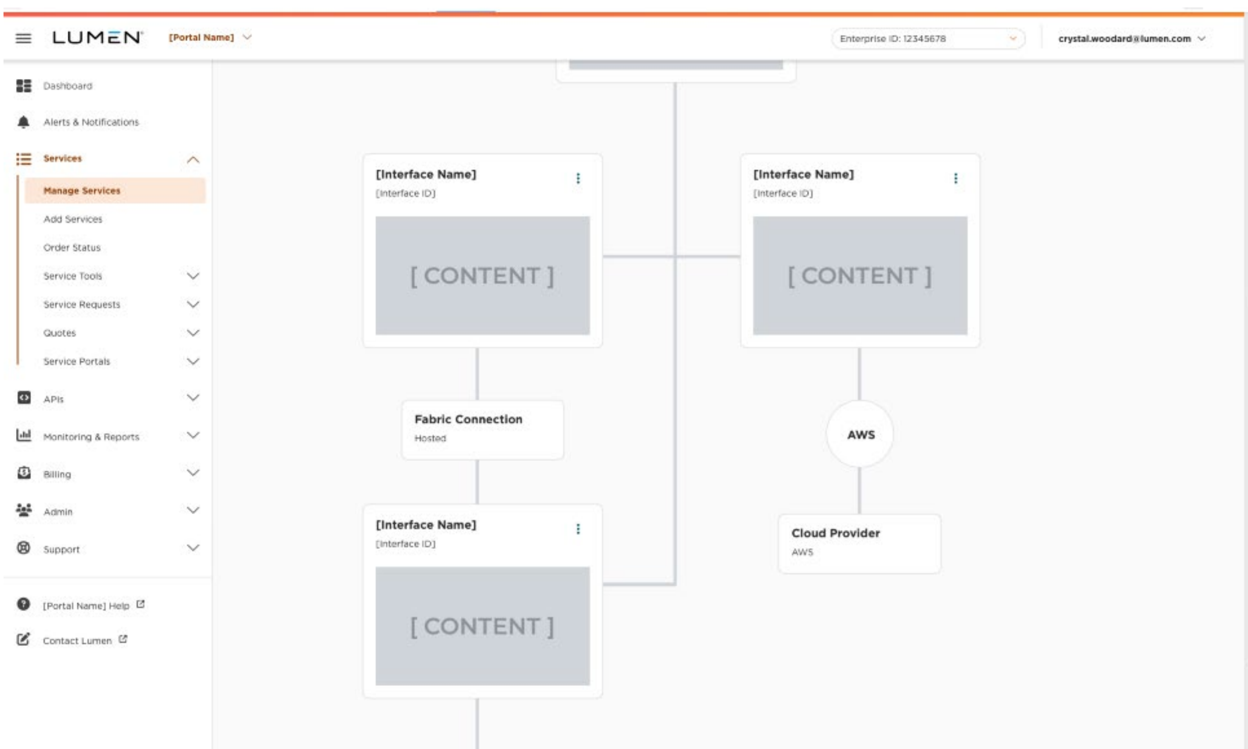
+-----------------------+---------------------+----------------+----------------------+--------------------+-----------
---------+-----+-----------------+-----------------+------------+

Service Details - Interface

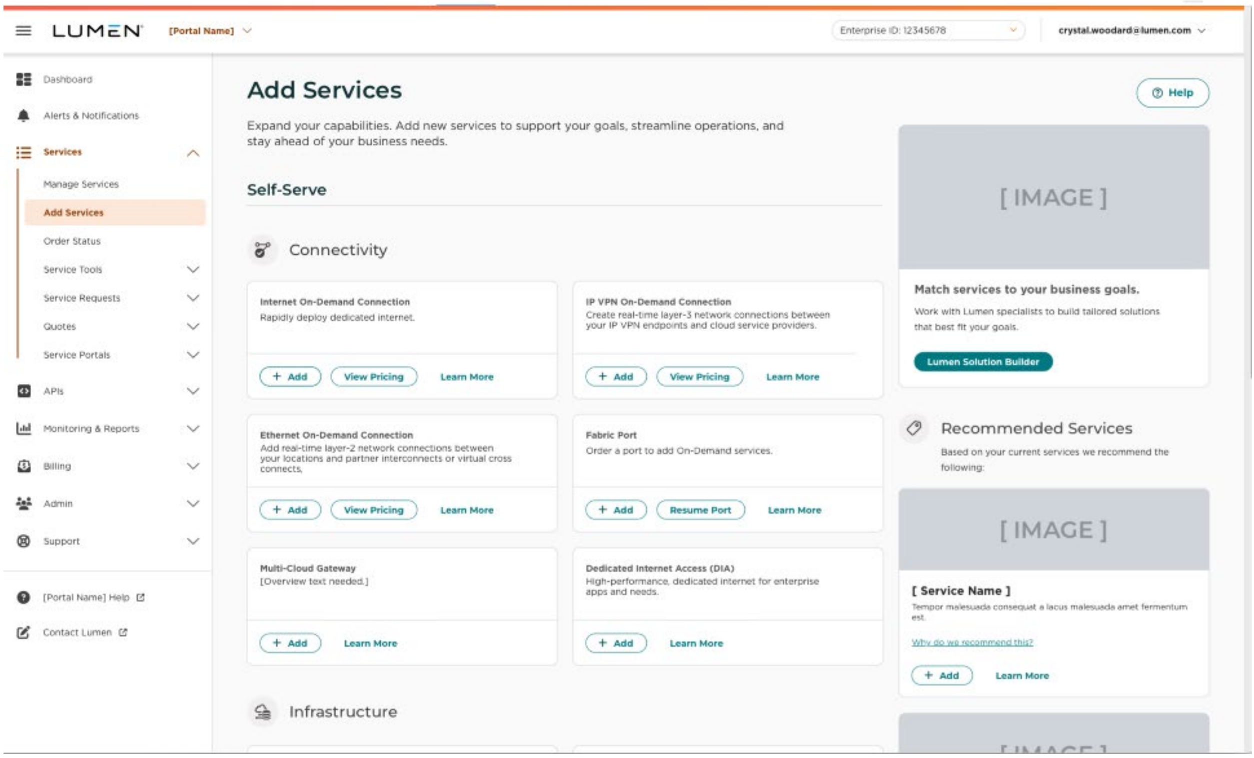


Service Details - Topology



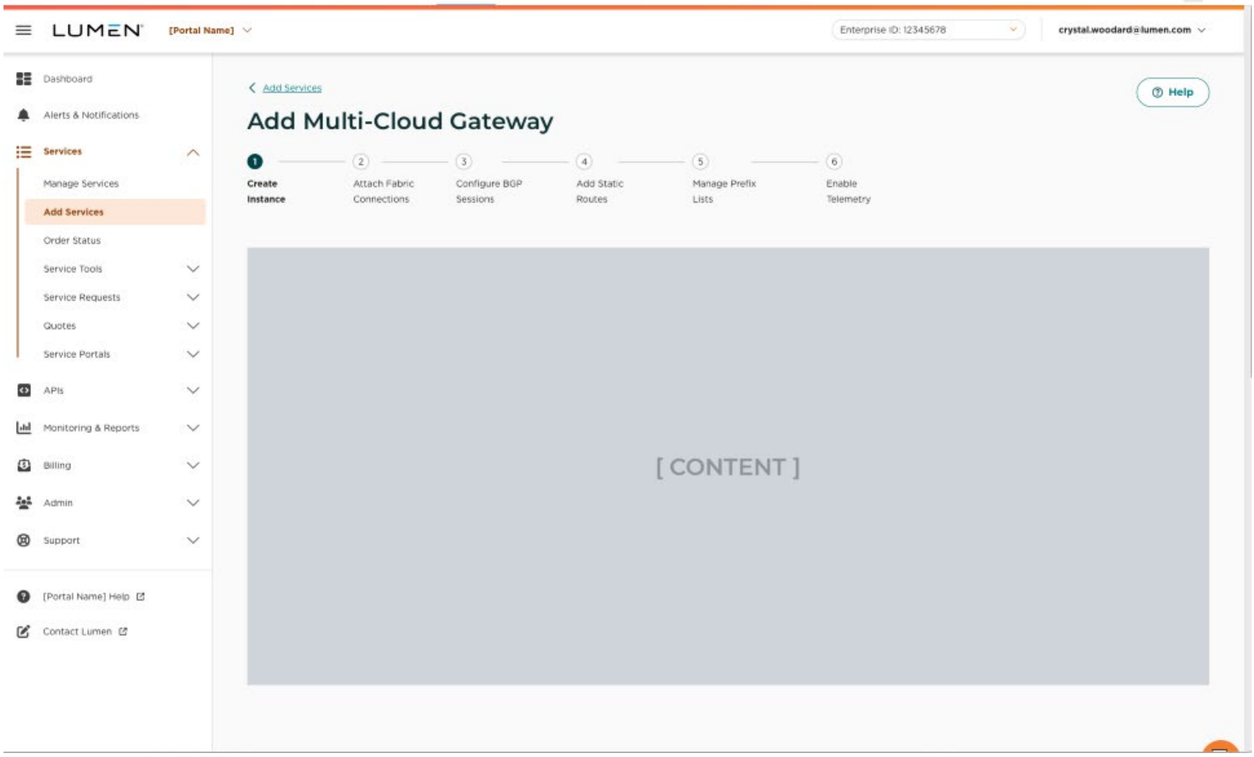


Add Services



- Current IPVPN process has ability to select "view pricing" or "learn more" which is a hyperlink to a datasheet for the product tiers. Lets keep that functionality as hinted at in ADD SERVICES wireframe above to ensure parity with existing services/UX

Add Services - Multi-Cloud Gateway Workflow



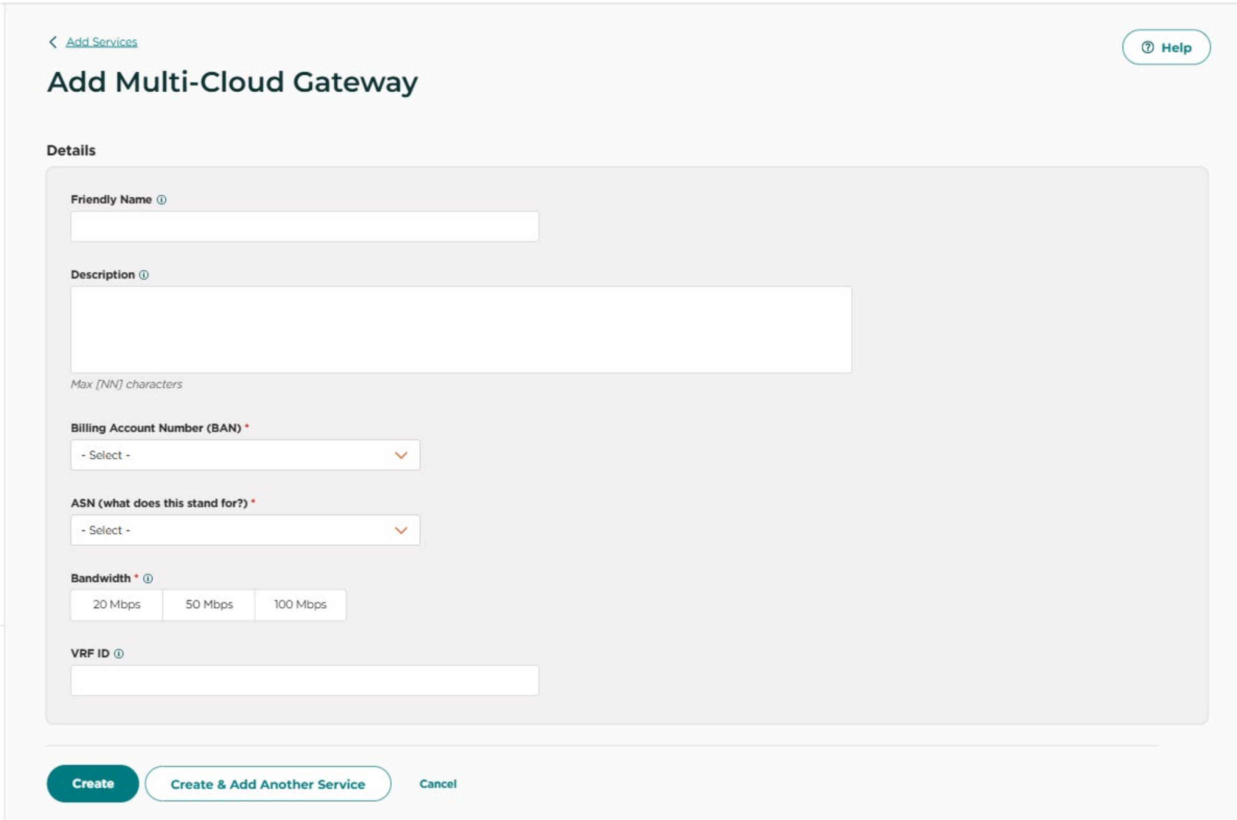
When a user starts the workflow to provision a Multi-Cloud Gateway the following should be the steps and options at each step of the workflow.

There should be a *"Cancel"* button at each step to allow a user to exit the workflow. When the button is clicked the user should be presented the following message:

> *"Please confirm you wish to cancel, this will result in all information being lost and no resources being created"*

Step 1 - Create MCGW Instance

(Option 1 from Wireframes appears to be most aligned with the UX we are seeking)



User should be provided the ability to create a MCGW instance by filling out the following fields (only name and tier should be required):

- Name ("friendly name" as shown in wireframe – nomenclature will be determined based on consistency across Control Center)
- Description
- Tier (selectable options)
  - Display the Tiers available along with pricing for each tier
  - pricing model proposal (9/3/25) – pricing convo ongoing

Up to 10Gbps -	$ 500	per month
Up to 50Gbps -	$ 750	per month
Unlimited -	$ 1,200	per month

- ASN (dropdown)
  - "ASN 1" should be the option that is displayed automatically.
  - User has ability to select drop down to change to "ASN 3549"
  - Create a hoverbox next to ASN to explain (The MCGW default Lumen ASN will be AS1. AS3549 will be an option used especially for existing customers that would already to peering with AS3549)
- VRF ID
  - Removed from wireframe with agreement by Chris

  - we assign the VRF ID for new builds and we are not supporting connections to existing IPVPN VRFs in MVP 1, so there is no VRF to specify at this time
  - consideration: building a VRF selection feature now and hiding it until future release

After selection of a tier the user should be presented the price of the instance.

Quantity	Service	Cost
1	Multi-Cloud Gateway Instance (tier level)	$0.00
		Total$0.00

The user should also be presented with the following options:

- Create Multi-Cloud Gateway Instance
- Add Multi-Cloud Gateway Interfaces

If *"Create Multi-Cloud Gateway Instance"* is selected and the *"Next"* button is clicked, a summary of the instance configuration based on what the information the user provided and selected in the first step along with the cost.

Multi-Cloud Gateway Instance

- Name
- Description
- Tier

Quantity	Service	Cost
1	Multi-Cloud Gateway Instance (tier level)	$0.00
		Total$0.00

There should be a *"Submit"* button at the bottom that a user would click that results in creation of the instance and the user being redirected to the "Order Processing" screen which displays status of the action and details of request.

Order Processing screen will display either success or failure of the provisioning request.

Upon successful instantiation of MCGW Instance, a button will be displayed on the screen with says "Add Interfaces"

Upon failure to instantiate MCGW Instance, a "back" button will be displayed to allow user to step backwards in the process, correct data if necessary, and retry the submit.

Successful instatiation of the MCGW Instance will return the following data:

```
{
  "mcgwId": "mcgws:5cceadea-0412-4ec7-ad31-a2de609b20bf",
  "name": "Production-MCGW-East",
  "asn": 1,
  "description": "Primary production gateway for east region cloud conne
ctivity",
  "tier": "100G",
  "term": "Hourly",
  "serviceId": "00/MCGW/025754/LUMN",
  "customer_number": "1-ABCD",
  "billing_account_number": "1-1ABCDE-F"
}
```


Failure to Instatiate the MCGW Instance has 7 possible error types. They will return the following data:

(Note: exact error messages to be displayed in TITLE field have not been defined yet)

Error Message “400 Bad Request”


```
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message “401 Unauthorized”

```
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message “403 Forbidden”


```
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message “404 Not Found”

```json
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message "422 Unprocessable Entity"


```json
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message "500 Internal Server Error"

```
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Error Message: "Default Error"


```
{
  "type": "/problems/uri-reference",
  "title": "some title for the error situation",
  "status": 599,
  "detail": "some description for the error situation",
  "instance": "/some/uri-reference#specific-occurrence-context",
  "errors": [
    {
      "detail": "string",
      "pointer": "string",
      "parameter": "string",
      "header": "string"
    }
  ]
}
```


Step 2 - Create MCGW Interfaces

User should be provided a text field to indicate the number of MCGW Interfaces the user would like to create.

- How many Multi-Cloud Gateway Interfaces would you like to create?

After entering the number of MCGW Interfaces the user should be presented with a list of interfaces based on the number the user entered. The following two text fields should be under each interface, and the user should be able to enter a value for each.

Interface X

- Name
- Description

Interface Y

- Name
- Description

The cost of the MCGW instance and Interface(s) should be shown at the bottom.

Quantity	Service	Cost
1	Multi-Cloud Gateway Instance (tier level)	$0.00
2	Multi-Cloud Gateway Interface	$0.00
		Total$0.00

The user should also be presented with the following options:

- Create Multi-Cloud Gateway Instance and Interface(s)
- Add Fabric Connection(s) to Multi-Cloud Interface(s)

Under the *"Add Fabric Connection(s) to Multi-Cloud Interface(s)"* option there should be a message indicating the following:

> *"This option requires the Multi-Cloud Gateway Instance and Interfaces to be created, and billing to start for the Instance. Billing will not start for any Interface until a Fabric Connection is attached to that Interface."*

If *"Create Multi-Cloud Gateway Instance and Interfaces"* is selected and the *"Next"* button is clicked, a summary of the instance configuration based on what the information the user provided and selected in the first step along with the cost.

Multi-Cloud Gateway Instance

- Name
- Description
- Tier

Multi-Cloud Gateway Interface

- Number of Interfaces

Quantity	Service	Cost
1	Multi-Cloud Gateway Instance (tier level)	$0.00
2	Multi-Cloud Gateway Interface	$0.00

	Total$0.00

Interfaces pricing tier model proposal (9/3/25) has 2 options – pricing convo is ongoing

Hourly -	$ 0.20
Monthly -	$ 100.00

There should be a *"Finish"* button at the bottom that a user would click that results in creation of the instance and the user being redirected to the service details page for the newly created instance.

If *"Add Fabric Connection(s) to Multi-Cloud Gateway Interface(s)"* is selected and the *"Next"* button is clicked, should result in moving the user to the next step (step 3). The instance nor the interfaces should not be created at this point.

Step 3 - Create Fabric Connections

Users should be provided a list of MCGW interfaces based on the number selected in the previous step. Each interface should have an *"Add Fabric Connection"* next to it on the right.

When a user clicks the *"Add Fabric Connection"* button, the user should be redirected to the second step in Fabric Connections workflow.

Modify A MCGW Instance

Modify A MCGW Interface

Phase 1 will leverage a GUI built on the go forward Global IT target architecture in order to present a customer experience commensurate with the long-term, target CX/UX. The GUI will be the front-end to a self-service platform and APIs which enables the customer to complete activities inclusive of:

- Order creation and management (MACD)
- MCGW feature selection (ie BGP, NAT)
- Creation and Modification of attached Cloud Connectivity, Fabric Connections/Ports
- Display of critical details for Layer 3 capable customer devices (IP, Subnet, ASN, etc)
- Order status tracking
- Historical activity tracking
- Service Inventory
- Fabric Port/Site inventory
- Reports demonstrating SLAs and service performance
- Support and maintenance section for trouble ticket creation/management and status
- Billing section
- Site admin section for role-based access control of authorized customer end users
- Knowledge base (or link to appropriate resource)

(Cloud Partner Portal Interactions including features like collection of pairing keys or API interactions to be defined in fabric connections/cloud connectivity PRD)

Ordering

1) Within the "Orders" section, allow customer to see historical order activity inclusive of pending, completed and cancelled orders.
2) Within the "Orders" section, allow customer to submit a New Start request
3) Within the "Orders" section, allow customer to submit a Change request
   a) Change activity is inclusive of upgrade/downgrade bandwidth and other activities outlined in Section 4.4
4) Within the "Orders" section, allow customer portal access to submit a Disconnect request and follow its status through completion or cancellation.

5) During order creation customer will utilize a guided workflow which queries the user for data dependent upon the selected action (new, change, disconnect)
   a) Bandwidth tier selection (Names bandwidth values - tbd)
      i) ~~Lab/Trial (Maximum – 2 connected endpoints, Bandwidth-TBD, 90 calendar days)~~
      ii) Up to 10Gb (Essentials)
      iii) Up to 50Gb (Plus)
      iv) Unlimited Bandwidth (Premium)
   b) Bandwidth Tier Metering – Pricing will be based on requested, aggregate bandwidth tiers. We will present the user a checkbox which enables them to turn feature on/off which approves auto-upgrades to higher tier when aggregate connected bandwidth exceeds requested tier. This will act as a safeguard for the customer who does not want to be billed for a higher tier for any reason or who wants to review an upgrade before proceeding.
      i) When aggregate bw usage exceeds the specified tier, system will trigger notification to the customer to review their tier assignment and decide if they want to approve upgrade to higher bw tier.
      ii) Name the bandwidth tiers
         ~~(1) Lab/Trial~~
         (2) Up to 10Gb (Essentials)
         (3) Up to 50Gb (Plus)
         (4) Unlimited Bandwidth (Premium)
   c) MCGW feature selection (ie BGP, NAT, etc as defined elsewhere in this PRD)
   d) VRF Name
   e) Site/Route creation and modification
   f) Assignment of Rate Limits and Class of Service (CoS) to assigned sites/routes
   g) Display of critical details for Layer 3 capable customer devices (IP, Subnet, ASN, etc)
   h) MCGW access zones will be selectable for redundancy/diversity management
6) Cloud Partner portal interaction will enable the customer to
   a) Enter critical configuration data including cloud location and pairing keys

Historical Activity

1. The “historical activity” section will display all pending, complete and cancelled MCGW service activities for the customer which may be unique from order activity (ie changing BGP details, renaming sites). This would optimally be a widget within a dashboard which users can interact with independently of other actions. (trish clay [peer of shac who ran control center] for shared service we can leverage – pull her team into conversation on digital buying experience for MCGW)

Service Inventory

1. The “service inventory” section will display the inventory of MCGW services the customer has in active state
2. This section is optimally designed to display a graphical representation of the network topology to enhance user understanding of their MCGW services. This may always display on screen as a topology or be displayed on-demand – tbd as UX evolves.

Site Inventory (fabric port inventory?)

1. The “site inventory” section will display the list of sites the customer has loaded into the platform for utilization with their MCGW service.
2. Sites will be selectable so that additional information displays upon selection indicating whether the site is in use or not and the appropriate details (CoS, rate limits, etc).
3. This may be inclusive of functionality (platform dependent) to have sites that are preloaded for future assignment to MCGW so that customers with complex networks can increase ease of use across user base.
4. Sites will have a “site name” feature enabling the customer to name their sites according to their internal practices.

Reports

1. Reports functionality will enable customer to pull historical reports on Network SLA performance

Support and Maintenance

1. “Support & Maintenance” section will allow customer to create a Trouble Ticket and follow its status through completion or cancellation.
2. “Support & Maintenance” section will allow customer to submit a Trouble Change request and follow its status through completion or cancellation.
3. “Support & Maintenance” section will allow customer to view the Network Maintenance calendar and related scheduled maintenance notification details.

4. "Support & Maintenance" section will allow customer to submit requests for GUI/Platform Support Ticketing and follow its status through to completion

Billing Section

1. "Billing" section will display MCGW services and sites which are currently being billed. The following details will be included in the display
    a. Start date
    b. End date
    c. Contract Term (Phase 1 may indicate "none" or "open")
    d. When charges are collected (ie start of month)
    e. Payment method(s)
    f. Link to or display of invoices
2. Ability to pay or dispute invoices and initiate questions to Lumen may not be in-scope dependent upon determination of go forward Global IT target architecture and its interaction with this platform.
3. Support the ability to toggle between the display of US Dollars (default) and native inter symbols

Site Administration

1. The "site administration" section will enable administration of role-based access control for customer users. Customers may have a series of users which they desire to grant access to the platform in order to create and manage orders, review billing history, create and manage Service Assurance tickets, or perform audits without having the ability to impact other data and activities.
2. This feature will be inclusive of appropriately predesigned user persona for each user type in-scope (tbd in alignment with roles for go forward CX/UX). Examples:
    a. System Administrator: CRUD access to all features and functionalities, including user admin.
    b. Service Designer: CRUD access to all features and functionalities on platform with the exception of user administration.
    c. Service Manager: Read Only access to all data, CRUD access to trouble ticketing features
    d. Service Operator: Read Only access to all data on the platform. No ability to modify data or create orders and tickets

Knowledge Base

1. The “knowledge base” will be a section or link to documentation which describes FAQ and M&Ps for MCGW services, GUI actions, Support & Maintenance information, and billing.

API

1. This is an API first initiative, the API should support all functionalities noted above in the event that the customer prefers to interact with MCGW platform programmatically via their tech stack
2. API adheres to OpenAPI 3.0 Specification

4.5.2 Seller-Led

Seller-led should not be supported, the service can only be provisioned through the API or UI. There will be no ability to order or provision outside the API and UI.

Future product enhancements where a fully managed version of MCGW would be made available could potentially be seller-led and quotable, but that is not part of the roadmap at this time.

4.5.3 Channel-Led

Channel-led will be evaluated in the future.

~~Customers will work with sales to understand the product offering, potential for bundling with other products and services, MCGW capabilities and pricing tiers.~~

~~Sales will own onboarding customer to Control Center and user training~~

~~Customers responsible for provisioning requests through GUI or API with Phase 1~~

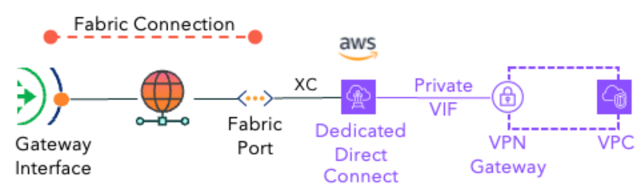
~~Phase 2~~

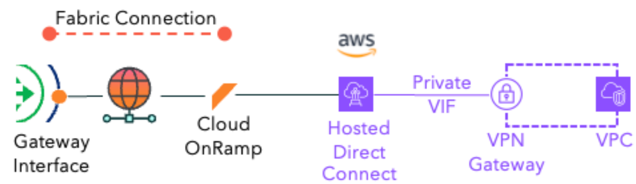
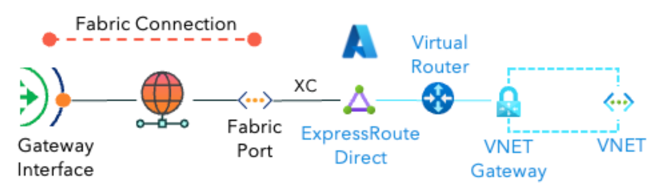
~~Sales will work with customer to determine if orders will be placed by sales on their behalf or by customer as self-service in the GUI.~~

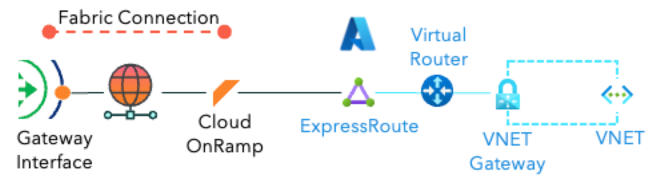
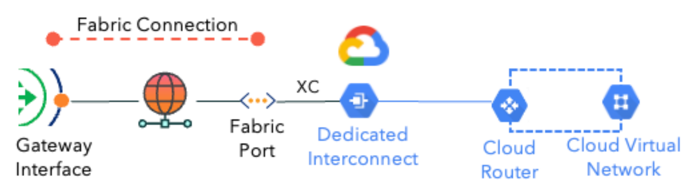
4.6 Product and Partner Integrations (PM)

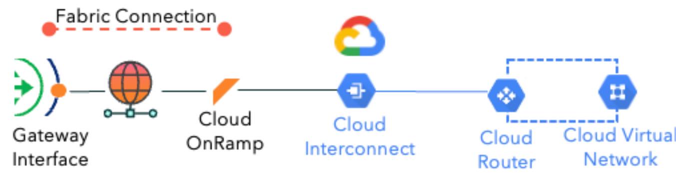
[Provide the products and partner solutions the product can integrate, list each supported product and partner, and describe how the integration would work. Indicate if a package could be created.]

Lumen Fabric Connections	Yes	Multi-Cloud Gateway will integrate with the Lumen Fabric Connections portfolio, with each MCGW Interface being able to home/attach to a Fabric Connection service
Lumen IP VPN	Yes	Multi-Cloud Gateway will integrate with the existing Lumen IP-VPN portfolio, with each MCGW Interface being able to privately route to an IP-VPN VRF

AWS Dedicated Direct Connect	**No**	High	There should be a **Cross-Connect** (XC) integration method defined for customers of Multi-Cloud Gateway, Fabric Connections and Fabric Port to tie into a Dedicated Direct Connect  This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/aws)
AWS Hosted Direct Connect	**No**	High	There should be a **Cloud OnRamp** integration method defined for customers of Multi-Cloud Gateway and Fabric Connections to tie into a Hosted AWS Direct Connect

			 This integration is referenced above in Feature ID MCGW-FC-02 Note: A **Cloud OnRamp** is a shared Layer 2 NNI with the cloud provider. Each OnRamp will be counted as a Fabric Port with Attribute Type equal to "Interconnect" This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/aws)
Azure ExpressRoute Direct	**No**	High	There should be a **Cross-Connect** (XC) integration method defined for customers of Multi-Cloud Gateway, Fabric Connections and Fabric Port to tie into an ExpressRoute Direct instance  This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/azure)
Azure ExpressRoute	**No**	High	There should be a **Cloud OnRamp** integration method defined for customers of Multi-Cloud Gateway and Fabric Connections to tie into a Hosted ExpressRoute

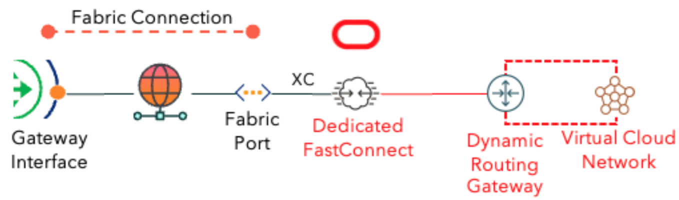
			 This integration is referenced above in Feature ID MCGW-FC-04 This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/azure) Note: A Cloud OnRamp is a shared Layer 2 NNI with the cloud provider. Each OnRamp will be counted as a Fabric Port with Attribute Type equal to "Interconnect"
Google Cloud Dedicated Interconnect	**No**	High	There should be a **Cross-Connect** (XC) integration method defined for customers of Multi-Cloud Gateway, Fabric Connections and Fabric Port to tie into a Google Cloud Dedicated Interconnect instance  This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/gcp)
Google Cloud Interconnect	**No**	High	There should be a **Cloud OnRamp** integration method defined for customers of Mult-Cloud Gateway and Fabric Connections to tie into a Hosted Cloud Interconnect

<table>
<tr><td></td><td></td><td></td><td>


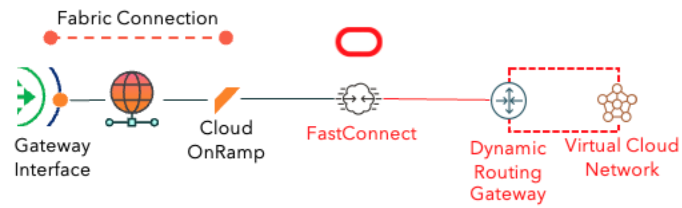
This integration is referenced above in Feature ID MCGW-FC-03

This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/gcp)

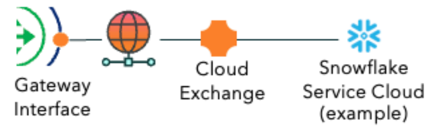
Note: A **Cloud OnRamp** is a shared Layer 2 NNI with the cloud provider. Each OnRamp will be counted as a Fabric Port with Attribute Type equal to "Interconnect"
</td></tr>
<tr><td>Oracle Cloud Infrastructure (OCI) Dedicated FastConnect</td><td>**No**</td><td>Low</td><td>
There should be a **Cross-Connect** (XC) integration method defined for customers of Multi-Cloud Gateway, Fabric Connections and Fabric Port to tie into a OCI Dedicated FastConnect instance

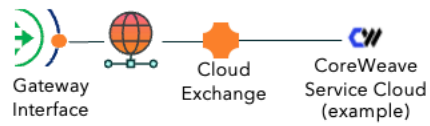


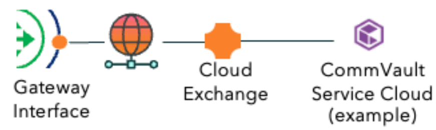
This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/oci)
</td></tr>
<tr><td>Oracle Cloud Infrastructure (OCI) FastConnect</td><td>**No**</td><td>Low</td><td>There should be a **Cloud OnRamp** integration method defined for customers of Multi-Cloud Gateway and Fabric Connections to tie into a Hosted FastConnect</td></tr>
</table>



			This integration is referenced above in Feature ID MCGW-FC-05 This integration would be documented and publicly posted to a site easily navigable from the rest of Lumen's technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/oci) Note: A **Cloud OnRamp** is a shared Layer 2 NNI with the cloud provider. Each OnRamp will be counted as a Fabric Port with Attribute Type equal to "Interconnect"
Service Cloud Exchanges	**No**	Low (Future Release)	There should be a **Cloud Exchange** integration method defined for customers of Multi-Cloud Gateway to make optimally-routed connections to different Service Clouds that do not have Layer-2 connectivity offers.







These integrations are referenced above in Feature ID MCGW-CX-01

These integrations would be documented and publicly posted to a site easily navigable from the rest of Lumen’s technical documentation (e.g. https://www.lumen.com/help/en-us/cloud/exchange)

NOTE: An **Cloud Exchange** is a shared Layer 3 route exchange with the service cloud. Each Exchange will be counted as a Fabric Port with Attribute Type equal to “Interconnect”

4.7 Product Model - Attributes (PM & EA)

[Provide details of any new and existing attributes that are required for the product. Indicate if the attribute is new within the heading.

Product Management and Enterprise Architecture should collaborate to complete this section]

The following are the list of new and existing attributes that are required by the product/feature.

4.7.1 Multi-Cloud Gateway Instance - New

Price Affecting - Yes

10G	Throughput of the provisioned instance.
50G	The value indicates total aggregate bandwidth tier of an instance.
Unlimited	

4.7.2 Multi-Cloud Gateway Interface - New

Price Affecting - Yes

1 - 1024	Virtual Interface that a Fabric Connection is attached to and where the Layer 3 addressing and BGP peering is configured. The value is the number of interfaces tied to an instance

4.7.3 Multi-Cloud Gateway NAT - New

Price Affecting - Yes

True	Network Address Translation (NAT).
False	The value indicates whether NAT is configured.

4.8 Quoting, Order Entry / Order Validation, and Workflow

Multi-Cloud Gateway will support platform-led sales only, it will not be a quotable service by a direct or indirect seller. The ability to provision this service outside of the API or UI will not exist. This is a consumption-based service that is intended to be transient in nature. While there will be some customers that will use the service in a more permanent nature, the potential to add term-based pricing in the future should be cared for. However, even if term-based pricing were added in the future, it still would not be a quotable service by a direct or indirect seller.

4.8.1 Quote to Cash

{Provide requirements for each category, add rows as needed]

MACD Rules

Finalize Attributes for Product Model

Zone-Based Pricing

Low	N/A for Phase 1 release Will revisit for Phase 2 or international expansion

Proration Rules

Low Phase#1 will include Flat Rate Month to Month billing initiated after activation and with at least two MCGI's connected. For initial launch, when MCGI's are activated, the first monthly charge is prorated for the number of days remaining in the billing period.

NRC Tables

Low TBD based on Fabric Port Standards

Competition doesn't charge install fee

Early Termination Fees

Low N/A for Phase 1, TBD additional phases

Term Discounts

Low N/A for Phase 1, TBD additional phases

SLA Penalties

Volume Discounts

Low	TBD in association with Fabric Connections/Fabric Ports

Bundle Discount

Low	TBD in association with Fabric Connections/Fabric Ports

Diversity / Redundancy Pricing

Low	TBD in association with Fabric Connections/Fabric Ports

CPE Pricing Implication

N/A

Customer Premise vs Data Center Pricing

N/A

Offnet Pricing

N/A

Tax and Regulatory Rules

MRC Tables

Interstate vs Intrastate Decision and Tax Rules

Premium SLA Pricing

Location Specific Rules

4.9 Documentation (PM)

{Provide details on what is required to ensure public facing documentation is built and/or updated.

Product Management is responsible for providing information pertaining to the documentation for the product.]

New Section called Multi-Cloud Gateway (MCGW)

New Subsection under Multi-Cloud Gateway called Overview

New Subsection under Multi-Cloud Gateway called Interfaces

New Subsection under Multi-Cloud Gateway called BGP with the following subsections underneath:

- Peering
- Prefix-lists
- Communities
- Policies

New Subsection under Multi-Cloud Gateway called Static Routes

New Subsection under Multi-Cloud Gateway called Telemetry

New Section called Multi-Cloud Gateway in API documentation

New Section called MCGW Interfaces in API documentation

New Section called MCGW BGP in API documentation

New Section called MCGW Prefix-Lists in API documentation

New Section called MCGW Communities in API documentation

New Section called MCGW Policies in API documentation

4.10 Requirements with Expected Cross Functional Impacts (PM & Others)

[Provide what is required from other teams/organizations.

All team members are responsible to provide information pertaining to what teams need to be included and engaged throughout the development of the product.]

The following are the requirements that have cross functional impacts.

Legal

TBD

Regulatory

Provide details around tax implications

- Are additional taxes required if MCGW utilizes any of the following products
  - Fabric Connections
    - Ethernet
    - Cloud
  - Fabric Services
    - Internet
    - MPLS
  - Fabric Ports
    - Customer Premise
    - Third Party Data Center
    - Lumen Data Center

Procurement

None

Security & Data Privacy

TBD

Customer Experience

???

Architecture & Engineering

- Provide analysis of the network around its ability to handle the additional functionality this product will require from it.
- Validation of the assumptions

Operations

Finance & Reporting

AGT Sales

AGT Partners	None
AGT Customer Success	None
Marketing	
Service Delivery	
Service Assurance	
Planning	
Migrations	None
Billing	
Other	

4.11 Quality Assurance and Testing (A&E)

[Provide details for any specific testing requirements.

Architecture and Engineering are responsible for providing the details on how the product and its functionality should be tested and validated]

4.12 Support Model (PM & Ops)

[Provide detailed requirements on how support for the product should be implemented and the priority of each.

Product Management is responsible to define the type and level of support required for the product.]

High	Support should be the same as VPN on-Demand (VoD) is today.

High With MCGW being a consumer of other products as listed below but not limited to going forward, thought needs to be done around matching support levels from an availability perspective across all products. If a customer requests support for MCGW and said investigation leads towards a Fabric Connection or Service issue. There can't be a bifurcated experience where the customer must wait for Fabric Connection or Service support due to being outside the available support hours.

- Fabric Connections
  - Cloud Hosted
  - Cloud Dedicated
  - Virtual Point-to-Point (EVPL - tagged service)
  - Transparent Point-to-Point (EPL - untagged service)
  - Virtual Multipoint-to-Multipoint (EVP-LAN - tagged ELAN service)
- Fabric Services
  - Internet
  - IP VPN

4.13 System and Technical Requirements (A&E and EA)

[System and technical requirements are the specific technical specifications a product or system must possess to meet its intended purpose, including performance standards, compatibility needs, and implementation details, essentially acting as a blueprint for developers to build the product effectively.

Architecture & Engineering, as well as Enterprise Architecture are responsible for providing the requirements pertaining to OSS/BSS and operational details for the product.

Example components of technical requirements:

Performance: *Processing speed, response time, data throughput.*

Security: *Data encryption, user authentication methods, access controls*

Integration: *How the system will interact with other systems or APIs*

Scalability: *Ability to handle increased user load or data volume*

Usability requirements: *Ease of navigation, intuitive interface design*

]

4.14 Non-Functional Product Requirements (PM)

[Non-functional requirements (NFRs) are product characteristics that describe how the product should perform, rather than what it does. They focus on qualities like reliability, security, performance, and usability.

Product Management is responsible for providing requirements for how the product should be expected to perform and exist.]

Performance	NFR-01	There should not be a throughput limitation, the throughput should be based on what the Lumen Fabric is able to support.

5 Pricing and Commercial Model

[Provide an overview of how the product will be priced.

If a Billing Component Matrix (BCM) exists, provide a link to the document.]

5.1.1 Go-to-Market (GTM) Pricing Strategy

[Product go-to-market pricing strategy refers to the pricing plan a company chooses to launch a new product in the market, considering factors like target audience, competition, product value proposition, and desired market penetration, all within a comprehensive go-to-market strategy that outlines how to reach customers and sell the product effectively.

Key aspects of a product go-to-market pricing strategy:

***Market analysis:** Understanding the current market pricing, competitor pricing, and customer price sensitivity to determine a suitable price point.*

***Value-based pricing:** Setting a price based on the perceived value of the product to the customer, highlighting unique features and benefits.*

***Cost-plus pricing:** Calculating the cost of production and adding a desired profit margin to arrive at a price.*

***Penetration pricing:** Introducing a product at a low price to quickly gain market share, potentially increasing prices later.*

***Price skimming:** Launching a product at a high price to capture early adopters and premium market segments, then gradually lowering the price.*

***Tiered pricing:** Offering different price points based on customer segments or purchase volume.]*

5.1.2 Competitive

https://www.megaport.com/products/megaport-cloud-router/

Launched in 2018 a "maturer" offering and probably one of MCGW's main competitors. Virtual router to handle traffic b/n two or more clouds at the edge for as low as $780 MRC. Pay as you go options available

https://www.consoleconnect.com/cloudrouter/

Launched a scaled down and limited version of Cloud Router in 2015, but the full solution that would directly compete with MCGW wasn't launched until 2022.

https://packetfabric.com/virtual-cloud-router

Launched in 2021.

https://www.equinix.com/products/digital-infrastructure-services/equinix-fabric/fabric-cloud-router

Launched in 2024. Newer offering under Equinix's Fabric Services architecture.

https://go2.digitalrealty.com/rs/087-YZJ-646/images/DOC_ServiceFabric_Virtual_Router_Infopaper.pdf

Digital Realty's ServiceFabric Virtual Router launched in 2022.

5.1.3 Models

This is a consumption-based service, as such a consumption-based pricing model will be used.

Pricing will be based on two key elements; these two elements are the fundamental building blocks of the Multi-Cloud Gateway service.

- Multi-Cloud Gateway Instance
  - Monthly
- Multi-Cloud Gateway Interface
  - Hourly
  - Monthly

Initially there will be three tier levels for an instance. Tiers will be based on total aggregate bandwidth, which is calculated by adding the bandwidth of each Fabric Connection attached to the instance. The cumulative total determines the aggregate bandwidth of a provisioned instance.

Multi-Cloud Gateway Instance Tiers

- 10G - Up to 10Gbps of aggregate bandwidth
- 50G - Up to 50Gbps of aggregate bandwidth
- Unlimited - Unlimited aggregate bandwidth

There should be a cost associated to each MCGW Interface that has a Fabric Connection attached to it. If a user provisions a MCGW Interface but does not attach a Fabric Connection to it, there should not be a cost associated.

A Fabric Connection attached to a MCGW Interface carries its own associated cost; it does not include the cost of a MCGW Interface.

Additional functionality beyond Layer 3 routing, for example Network Address Translation (NAT), will be priced on an individual basis. This will allow future functionality to be easily added and priced based on a single set or multiple sets of criteria.

Examples:

v

5.1.4 Pricing

MCGW Instance Tiers

- 10G - Up to 10Gbps - $500.00/month
- 50G - Up to 50Gbps - $750.00/month
- Unlimited - Unlimited - $1,200.00/month

MCGW Interface

- Hourly - $0.20
- Monthly - $100.00

5.1.5 Special Pricing

There are no special pricing requirements.

6 Product Release

The following sections detail the plan and strategy for releasing the product/feature and its growth.

6.1 Market Plan

[Product market plan is a strategy that outlines how a company will introduce a new product to the market.

Provide a plan for bringing the product to market.]

6.1.1 Market Growth Drivers

[Product market growth drivers are the key factors that contribute to the expansion and success of a product within its target market, including elements like strong customer demand, emerging trends, technological advancements, effective marketing strategies, competitive differentiation, and a good product-market fit, all working together to drive sales and market penetration.

Example market growth drivers:

***Market Analysis and Customer Insights:** Understanding customer needs, preferences, and pain points through market research to tailor products accordingly.*

***Product-Market Fit:** Aligning the product features and benefits with the specific needs and desires of the target market.*

***Innovation and Continuous Improvement:** Regularly developing new features, functionalities, or product iterations to stay ahead of competition and meet evolving customer demands.*

***Effective User Acquisition Strategies:** Implementing targeted marketing campaigns and distribution channels to reach potential customers and drive adoption.*

***User Retention and Engagement:** Focusing on strategies to keep customers actively using the product and maximizing their lifetime value.*

***Data-Driven Decision Making:** Utilizing data analytics to monitor performance, identify trends, and make informed decisions about product development and marketing.*

***Scalability and Infrastructure:** Having the capacity and systems to support growing customer base and market expansion.*

***Strategic Partnerships:** Collaborating with other companies to access new markets, customer segments, or distribution channels.*

***Emerging Technologies:** Leveraging new technologies to create innovative products or enhance existing functionalities.*

***Competitive Differentiation:** Identifying unique selling points that set the product apart from competitors.*

]

6.2 Sales Plan (PM, Segments, AGT)

[Product sales plan is a roadmap that outlines how to generate revenue from a new product. It includes goals, strategies, tactics, and the people involved.

Key aspects of a product sales plan:

***Goals:** The revenue targets and objectives for the product*

***Target market:** Who will buy the product and their needs*

***Competitive analysis:** How the product compares to competitors*

***Sales strategies:** How to sell the product and attract new customers*

***Sales tactics:** Specific actions to take to achieve sales goals*

***Sales enablement strategy:** How to launch the product and generate revenue*

***Budget:** How much to spend on sales and marketing*

]

6.3 Release Criteria

- Fabric Port needs to be released prior to MCGW
- Fabric Connections needs to be released prior to MCGW
- Public facing APIs need to be redesigned prior to MCGW

6.4 Schedule and Drivers

[Provide details on the release schedule and what is driving the schedule]

Leadership has requested MCGW to be launched beginning to middle of September 2025.

6.5 Key Milestones

[Provide the key milestones and their respective timelines]

7 Product Lifecycle

The following sections outline and describe how the product will mature across the multiple stages of the product lifecycle.

1. Development
2. Introduction
3. Growth
4. Maturity
5. Decline

7.1 Development

This is the initial stage where the product concept is refined, tested, and a launch strategy is developed. The focus is on understanding market needs, designing the product, and ensuring it meets customer expectations. This stage involves research, prototyping, and testing to ensure the product is viable and meets the requirements.

[Provide additional details if any that were not included in the previous sections regarding this stage of the product's lifecycle.]

7.2 Introduction

The introduction phase is about establishing the product's core value proposition, building awareness, and gaining initial traction in the market. Marketing efforts are geared towards informing potential customers about the product and its benefits. Pricing strategies are often aggressive, aiming to capture market share.

[Provide additional details if any that were not included in the previous sections regarding this stage of the product's lifecycle.]

7.3 Growth

The product gains popularity and market share as customer adoption increases. Marketing efforts shift towards reinforcing the product's value proposition and building brand loyalty.

[Provide details around each of the following strategic areas as they relate to the product when reaching and in this stage of the product's lifecycle.]

7.3.1 Development Strategy

7.3.2 Marketing Strategy

7.3.3 Competitive Strategy

7.3.4 Go-to-Market (GTM) Strategy

7.3.5 Pricing Strategy

7.4 Maturity

The product reaches its peak market penetration, and sales growth begins to slow down. Competition intensifies, and pricing pressures may increase. Marketing efforts focus on maintaining market share and attracting new customers.

[Provide details around each of the following strategic areas as they relate to the product when reaching and in this stage of the product's lifecycle.]

7.4.1 Development Strategy

7.4.2 Marketing Strategy

7.4.3 Competitive Strategy

7.4.4 Pricing Strategy

7.5 Decline

The product's market share starts to decline as demand wanes and newer products emerge.

[Provide details around each of the following strategic areas as they relate to the product when reaching and in this stage of the product's lifecycle.]

7.5.1 Go-to-Market (GTM) Strategy

7.5.2 Development Strategy

7.5.3 Marketing Strategy

7.5.4 Competitive Strategy

7.5.5 Go-to-Market (GTM) Strategy

7.5.6 Pricing Strategy

7.5.7 Migration Strategy

7.5.8 MACD Strategy

7.5.9 Shutdown Strategy

8 Appendix

8.1 Product Requirements Process

The product requirements document (PRD) has been called the product manager's best friend. It serves as a compass, roadmap, and GPS in helping guide your product to launch. Give your PRD its due attention and it will become your trusted assistant!

The PRD is a living document to which everyone on your development team contributes and refers to, throughout the development process.

The product requirements document defines:

- The purpose and value proposition of a product to be developed
- The features and functionality (i.e., the requirements) for the product
- The goals and timeline for the product's release

Do Your Homework

To create a great new product, you need a thorough understanding of the problem that product is going to solve. The only way to gain that understanding is by doing some homework.

Study your customers. Talk to them. Discover exactly what problem they need you to solve for them. Why does it matter to them? What is it costing them?

Study your competitors. Study their solutions to the problem. From what you've learned from your customers, determine your competition's strengths and weaknesses when it comes to solving the problem. How can you better fill the need?

Consult with marketing, sales, and your technical experts, anyone who has insight into the problem.

Assess your team's capabilities. How are they equipped to tackle the problem? What technologies are they familiar with or excited about that might be useful in addressing the problem?

Study the available technologies. What are their advantages and disadvantages relative to solving the problem?

Define Your Products Purpose

You need to establish a clear, concise ***value proposition*** that succinctly communicates the need the product will fill and spells out how the product aligns with your company's overall objectives and product strategy. Hone this value proposition down to an "elevator pitch" that can be recited in under one minute.

Provide guidance that will help the team make tradeoffs during the design and implementation of the product. Make it clear to everyone what a successful implementation of the product will deliver.

Define Your Products Principles

Establish a set of principles for the product that will guide the team through development.

For a medical device, for example, the principles might be:

- Extremely safe
- Highly reliable
- Easy and intuitive to use

You may find that defining your product's principles leads you to refine and tighten up your value proposition statement. Or you might find that your principles fall simply fall out of the value proposition exercise. Either way, your product principles will act as a compass that will help guide your team as they define user tasks and product features in the next two steps.

Identify Customer and User Profiles, Goals, and Tasks

Once you have defined the purpose of your product, you need to establish who you're building it for. You need a clear picture of your product's users, their goals in using the product, and the tasks they will perform with the product to accomplish those goals. This is a drill-down exercise: First define the user profile, then the goals, and finally, the tasks.

Build Your Customer and User Profiles

As you begin to define your customer and user profiles, you'll probably be able to identify several potential customers and users for your product. Out of these, you need to identify the primary customer and user: the ones who are most important to your company in terms of sales of the product.

Once you've determined the primary customers and users, build a descriptive customer profiles and user profiles or personas. These profiles will indicate the target customer's and user's industry, segment, job function, and other data. It should also include any habits the customers and users have that might influence their use of your product.

Your goal in building customer and user profiles is to gain a collective understanding of where your primary customers and users are coming from. When your team considers new features, they should ask themselves how these customers and users would react to them and how likely they are to use them.

It's important to make your customer and user profiles as realistic and representative as you can. Team members need to visualize the customers and users. These profiles should also represent the needs, desires, habits and attitudes of a solid cross-section of your primary customers and users, so that you end up shaping your product for most of your primary customer and user groups and not some small niche within it.

Above all, be sure to focus only on the primary customers and users. Keep secondary customers and users out of the discussion. It is impossible to please everyone, so don't even try. Optimize for your primary customers and users.

Identify Customer and User Goals

Once you have your primary customer and user profiles, identify their main objectives in using the product. What do they need and want to accomplish?

Be aware that when you speak to customers and users, they may tell you the solution they think they want rather than their goals. Customers and users often have a tough time disentangling what they want to accomplish from how they go about accomplishing it. This is important, because there are likely much better ways to solve their problem than what they have in mind.

You need to give designers and engineers freedom to find the best solution. Keep your focus on determining exactly what the customers and users need and want to accomplish and what obstacles are standing in their way.

Identify Customer and User Tasks

Finally, work with your team to design tasks to help customers and users accomplish their goals.

Encourage creativity and innovation during this exercise. Try to envision ways they can accomplish their goals quickly and easily. Then, evaluate alternatives against one another.

Note: tasks are not features. Features support tasks and are defined later. Features should map through tasks to higher-level goals.

Specify Your Products Features

Now, your team is going to start filling in what will be the bulk of your PRD. In this section, you will describe each feature in terms of its functionality. You will describe any constraints that have been placed upon the design of the product. You should also note any assumptions made while defining your requirements.

The product's **functionality** will be expressed in what are known as **functional requirements.** Functional requirements dictate what the product must do. They must not, however, dictate how the product does that. The "how" will be determined during product design and development.

It is important that the requirements not dictate the implementation or bias it toward one solution or another. Only with "solution-agnostic" requirements will your designers and engineers be free to find a solution that is optimized toward accomplishing the user's goals.

The product's **constraints** will be expressed in what are known as **non-functional requirements.** Non-functional requirements capture any restrictions or limitations that have been placed on the product's design by stakeholders.

You may find it helpful to structure the description of each feature by breaking it up into parts within your template. Your "feature template" might include:

- Description
- Purpose

- User problem and task(s) the feature addresses
- Functionality
- Constraints
- Assumptions
- Design (preliminary wireframes or mockups)
- Not doing (anything not to be included in the release)
- Acceptance criteria

Finally, go back and question your assumptions. Have you fully identified all the assumptions you've made? Are those assumptions valid? Have forgotten anything? It is very easy to make assumptions without even being aware of them. Review your requirements to make sure they're solution-agnostic.

Test Your Concept

Product managers and developers often make the mistake of having too much confidence in their PRD once they've finished drafting it. The problem is that by the time customer and Beta feedback arrives, it's too late to make major changes.

The remedy for this problem is to perform product validation testing with customers.

Product validation testing is typically broken down into three types: feasibility testing, usability testing, and product concept testing.

Feasibility Testing

Feasibility testing involves building a prototype or model and then examining it to determine whether it is feasible to build.

Get your engineers, architects, and designers involved. Explore available technologies and possible approaches. Identify obstacles and evaluate their severity. Are they insurmountable? It's better to know now.

Usability Testing

Usability testing is a highly effective way to get feedback from your target customer. It often identifies missing requirements or requirements that are less necessary than originally supposed.

You and your designers will need to come up with ways of presenting the functionality so users can understand how to use the product. Plan on running multiple iterations before you arrive at a satisfactory user experience.

Product Concept Testing

Knowing your product is feasible and usable is not enough. What's most important is to make sure customers will actually buy it. For that, you need to get your product in front of customers and determine if it actually helps them accomplish their goals in a satisfying way. That's where product concept testing comes in.

Product concept testing can normally be combined with usability testing. To accomplish this goal, however, it's very important that the prototype be highly realistic. Fortunately, today's prototyping tools make this not only possible, but fast and easy as well.

Fold In Your Feedback

As you test your product concept, update your requirements based on the feedback you've gained. **The importance of this step cannot be overemphasized**. Most engineering errors originate in the requirements, and the cost of correcting those errors rises dramatically once implementation begins.

Establish Release Criteria and Timelines

Once you've established and validated your requirements, it's important to prioritize and rank them and to set your goals for the release.

Many product managers prioritize features by tagging them with labels, such as "must have," "high want," and "nice to have." But it's also important to rank order each requirement within those categories. There are two reasons for this.

The first is time to market. Schedules often slip. When they do, you may need to trim features and requirements to meet your release date. You don't want your team implementing the easiest requirements first only to find that there's not enough time to complete all your must-haves.

The second reason is that requirements evolve. During implementation, you're likely to discover new needs. Some may be critically important and supersede existing requirements in terms of priority. You need to know where those new requirements fit in with your pecking order. If you don't, less important factors will determine what gets implemented first, which may have a negative impact on your product's success.

Besides prioritizing requirements, you should also establish some quantifiable metrics for success that will establish the fitness of your product for release. Your success metrics may include measurements for:

- Revenue
- Performance
- Reliability
- Usability
- Supportability
- Scalability
- Other factors relevant to your product

Review and Revise Your Draft PRD

Before you start implementation, test your PRD for completeness.

Get all your stakeholders involved. They should make sure the PRD addresses their needs and concerns. Do the developers fully understand the problem to be solved? Does QA have enough information to build a test plan and write test cases?

Once you've addressed any issues your stakeholders have identified, you'll have a PRD from which you can begin to build a product.

Manage Product Development

Over the course of product development, countless issues over requirements (or the lack thereof) will arise. Always refer to the PRD. If the answer isn't there, resolve the issue and record the decision.

Your PRD is a living document. Use it to track all your features and requirements throughout development and product launch.

Above all, keep it accurate. Doing so will simplify preparation for milestone reviews. Plus, your whole team will have a clear picture of what you're all aiming for.